LAPORAN TUGAS BESAR 1

IF3070 DASAR INTELEGENSI ARTIFISIAL

PENCARIAN SOLUSI PENEMPATAN KENDARAAN PADA GELADAK KAPAL DENGAN LOCAL SEARCH



Disusun oleh:

Ahmad Aditya Ar Rasyid	18224023
Daniel Benaya Toar Supaath	18224047
Ahmad Rizal Fahmi	18224085

Sekolah Teknik Elektro dan Informatika

Institut Teknologi Bandung

2026

DAFTAR ISI

DAFTAR ISI **2**
Deskripsi Persoalan **4**
1. Latar Belakang dan Tujuan 4
2. Data dan Representasi Persoalan 4
3. Representasi Solusi 5
Pembahasan **8**
1. Representasi Solusi dan Fungsi 8
1.1. Model Data 8
1.2. Pemeriksaan Constraints 9
1.3. Perhitungan Nilai Objektif 12
1.4. Pembentukan Initial State 13
1.5. Operasi Neighbor 14
2. Implementasi dan Evaluasi Hill-Climbing with Sideways Move 16
2.1. Menghasilkan neighbour: get_neighbors() 17
2.2. Inisialisasi Pencarian: hill_climbing() 19
2.3. Pemilihan Tetangga Terbaik 20
2.4. Penerimaan Perpindahan dan Sideways Move 21
2.5. Kondisi Berhenti dan Hasil Pencarian 22
2.6. Keseluruhan Kode 23
3. Implementasi dan Evaluasi Simulated Annealing 27
3.1. Inisialisasi State Awal 27
3.2. Pencarian Neighbor dan Local Optimum 29
3.3. Evaluasi Kandidat dan Penerimaan Solusi 30
3.4. Pencatatan Hasil dan Penghentian Algoritma 30
3.5. Keseluruhan Kode 31
4. Implementasi dan Evaluasi Genetic Algorithm 37
4.1. Representasi Individu dan Populasi 37
4.2. Inisialisasi Populasi 38
4.3. Evaluasi Fitness 39
4.4. Seleksi Induk (Parent) 39

4.5. Crossover Satu Titik 39
4.6. Mutasi Individu 40
4.7. Pembentukan Generasi dan Penyimpanan Solusi Terbaik 42
4.8. Pencatatan dan Penyimpanan Hasil 45
5. Hasil dan Analisis Eksperimen 50
Kesimpulan dan Saran 100
1. Kesimpulan 100
2. Saran 101
Pembagian Tugas 102
Referensi 103
Lampiran Form Penggunaan Generative AI 104
Lampiran 106

Deskripsi Persoalan

1. Latar Belakang dan Tujuan

PT Owen Intim Line mengoperasikan kapal kargo untuk mengangkut kendaraan dari beberapa dealer. Kendaraan yang akan dikirim memiliki dimensi, berat, tarif pengiriman, dan perkiraan waktu kedatangan yang berbeda. Perbedaan tersebut membuat pemilihan muatan dan penataan kendaraan perlu mempertimbangkan ruang geladak serta kapasitas angkut kapal.

Penataan kendaraan secara manual dapat menyisakan ruang yang sulit dimanfaatkan dan menghasilkan pilihan muatan dengan keuntungan yang kurang baik. Kendaraan dengan tarif tinggi juga belum tentu dapat dimasukkan karena ukuran, berat, atau susunan kendaraan lain. Oleh karena itu, persoalan yang diselesaikan mencakup pemilihan kendaraan yang diangkut, koordinat penempatannya, dan orientasi setiap kendaraan.

Tujuan penyelesaian adalah mencari susunan kendaraan dengan total tarif pengiriman sebesar mungkin, dengan tetap memenuhi batasan ruang dan berat. Program menggunakan pendekatan local search untuk mengevaluasi kandidat solusi. Algoritma yang menjadi cakupan tugas adalah salah satu varian Hill-Climbing, Simulated Annealing, dan Genetic Algorithm.

2. Data dan Representasi Persoalan

Data persoalan terdiri atas satu kapal dan sejumlah kendaraan. Geladak serta kendaraan dimodelkan sebagai persegi atau persegi panjang pada grid dua dimensi. Dimensi menggunakan satuan sel grid, tarif pengiriman menggunakan satuan juta rupiah, dan berat menggunakan satuan unit sebagaimana ditetapkan pada spesifikasi.

Entitas	Atribut	Keterangan
	id	Identitas kendaraan yang

Kendaraan		unik
	dimensions	Lebar w dan panjang l kendaraan
	shipping_fee	Tarif pengiriman yang menambah nilai objektif apabila kendaraan diangkut
	weight	Berat kendaraan yang diperhitungkan dalam kapasitas angkut
	eta	Perkiraan waktu kedatangan dalam sisa hari
Kapal	dimensions	Lebar w dan panjang l geladak
	max_capacity	Batas total berat kendaraan yang dapat diangkut

Orientasi merupakan bagian dari penempatan kendaraan pada suatu solusi. Kendaraan dapat ditempatkan secara horizontal atau vertikal. Pada orientasi vertikal, lebar dan panjang efektif kendaraan dipertukarkan. Atribut eta tetap disimpan pada model kendaraan, sedangkan objective function dan constraints wajib yang digunakan dalam implementasi ini berkaitan dengan tarif, posisi, dimensi, dan berat.

3. Representasi Solusi

Satu solusi merepresentasikan keadaan seluruh kendaraan. Kendaraan yang diangkut mempunyai koordinat bilangan bulat dan orientasi. Kendaraan yang tidak diangkut diberi nilai None. Representasi tersebut memungkinkan program memilih sebagian kendaraan tanpa menghapus kendaraan lain dari data persoalan.

Implementasi menggunakan daftar positions dengan panjang yang sama dengan daftar vehicles. Elemen positions[i] selalu menyatakan penempatan vehicles[i]. Hubungan berdasarkan indeks dipertahankan selama pencarian

sehingga kendaraan tetap dapat diidentifikasi setelah perpindahan, pertukaran penempatan, maupun operasi genetik.

Koordinat x mengukur posisi sepanjang lebar geladak dan koordinat y mengukur posisi sepanjang panjang geladak. Kendaraan pada posisi (x, y) dengan dimensi efektif (w, l) menempati sel dengan koordinat x sampai x + w - 1 dan y sampai y + l - 1. Dua kendaraan dapat bersentuhan pada tepi selama tidak menempati sel yang sama. Pemetaan sel geladak dapat diturunkan dari koordinat dan dimensi efektif setiap kendaraan.

4. Fungsi Objektif

Nilai objektif suatu solusi merupakan penjumlahan tarif pengiriman seluruh kendaraan yang diangkut. Definisikan z_i(s) bernilai 1 apabila kendaraan ke-i diangkut pada solusi s dan 0 apabila kendaraan tersebut berada di luar kapal. Fungsi yang dimaksimalkan adalah sebagai berikut.

$$f(s) = \sum_{i=1}^{n} z_i(s) \cdot fee_i$$

Pada persamaan tersebut, (n) merupakan jumlah kendaraan dan (fee_i) merupakan tarif pengiriman kendaraan ke-(i). Kendaraan dengan penempatan None tidak menambah nilai objektif karena tidak diangkut. Nilai objektif digunakan untuk mengevaluasi kualitas solusi, dengan nilai yang lebih tinggi menunjukkan total keuntungan pengiriman yang lebih besar.

5. Batasan Solusi

Solusi yang diterima harus memenuhi batasan berikut.

a. Kendaraan yang diangkut berada sepenuhnya di dalam batas geladak. Koordinatnya tidak negatif dan batas kanan serta batas akhirnya tidak melewati dimensi kapal.

b. Kendaraan yang diangkut tidak saling bertumpuk. Setiap sel geladak hanya dapat ditempati oleh satu kendaraan.

c. Jumlah berat kendaraan yang diangkut tidak melebihi max_capacity pada kapal. Total berat yang tepat sama dengan kapasitas masih diperbolehkan.

d. Kendaraan yang tidak diangkut sepenuhnya berada di luar kapal dan direpresentasikan dengan None.

Data persoalan yang digunakan diasumsikan memiliki ID kendaraan unik, dimensi grid berupa integer positif, dan nilai tarif serta berat yang sesuai dengan domain persoalan. Pada pembentukan posisi secara random, orientasi diambil dari enum Orientation. Asumsi data tersebut menjadi prasyarat penggunaan fungsi inisialisasi dan evaluasi.

Pembahasan

1. Representasi Solusi dan Fungsi

Komponen bersama berada pada direktori src/init. Modul tersebut menyediakan model data, pemeriksaan *constraints, objective function*, pembentukan *initial state*, dan operasi *neighbor.*

1.1. Model Data

Model data didefinisikan dalam models.py. Orientation merupakan enum dengan pilihan HORIZONTAL dan VERTICAL. Dimension menyimpan lebar dan panjang. Vehicle menyimpan identitas, dimensi, tarif pengiriman, berat, dan eta. Ship menyimpan dimensi geladak serta batas berat muatan. Position menyimpan koordinat dan orientasi kendaraan.

Position menggunakan dataclass dengan frozen=True sehingga atribut suatu posisi tidak dapat diubah secara langsung. Perubahan penempatan dilakukan dengan membuat objek Position baru pada daftar solusi yang baru. Sifat ini juga memungkinkan daftar individu disalin secara dangkal ketika menyimpan populasi awal atau solusi terbaik.

Berikut source code model data pada src/init/models.py

```
from enum import Enum
from dataclasses import dataclass

class Orientation(Enum):
  HORIZONTAL = "horizontal"
  VERTICAL = "vertical"

@dataclass
class Dimension:
  w: int
  l: int

@dataclass
```

```
class Vehicle:
  id: str
  dimensions: Dimension
  shipping_fee: float
  weight: float
  eta: int

@dataclass
class Ship:
  dimensions: Dimension
  max_capacity: int

@dataclass(frozen=True)
class Position:
  x: int
  y: int
  orientation: Orientation
```

1.2. Pemeriksaan Constraints

Modul constraints.py menyediakan lima fungsi. direction_of_car mengembalikan dimensi efektif berdasarkan orientasi. is_overlapping memeriksa perpotongan dua persegi panjang. is_in_ship memeriksa batas penempatan. is_overweight membandingkan jumlah berat dengan kapasitas kapal. is_valid_state menggabungkan pemeriksaan tersebut pada suatu solusi.

is_valid_state terlebih dahulu memeriksa kesamaan panjang daftar kendaraan dan posisi, keunikan ID kendaraan, serta tipe integer pada koordinat kendaraan yang diangkut. Selanjutnya, fungsi memeriksa batas geladak dan overlap antarkendaraan. Kendaraan dengan posisi None dilewati. Berat hanya dijumlahkan untuk kendaraan yang diangkut.

Pemeriksaan overlap menggunakan batas persegi panjang. Apabila batas kanan salah satu kendaraan tidak melewati batas kiri kendaraan lain, atau hubungan serupa berlaku pada sumbu y, kedua kendaraan

dianggap terpisah. Penggunaan operator <= membuat kendaraan yang bersentuhan pada tepi tetap diperbolehkan.

Berikut source code constraint pada src/init/constraints.py

```
from .models import Dimension, Vehicle, Ship, Position, Orientation
from collections.abc import Sequence

def direction_of_car(vehicle: Vehicle, position: Position) -> Dimension:
        """Mendapatkan dimensi kendaraan berdasarkan orientasi posisinya"""
    if position.orientation == Orientation.VERTICAL:
                    return Dimension(w = vehicle.dimensions.l, l = vehicle.dimensions.w)

    return vehicle.dimensions

def is_overlapping(vehicle1: Vehicle, position1: Position, vehicle2: Vehicle, position2: Position) -> bool:
    """Mengecek apakah dua kendaraan saling bertumpuk"""
    dim1 = direction_of_car(vehicle1, position1)
    dim2 = direction_of_car(vehicle2, position2)

    if (position1.x + dim1.w <= position2.x or
        position2.x + dim2.w <= position1.x or
        position1.y + dim1.l <= position2.y or
        position2.y + dim2.l <= position1.y):
        return False

    return True

def is_in_ship (vehicle: Vehicle, position: Position, ship: Ship) -> bool:
    """Mengecek apakah kendaraan berada di dalam kapal"""
    dim = direction_of_car(vehicle, position)

    if (position.x < 0 or
```

```
            position.y < 0 or
            position.x + dim.w > ship.dimensions.w or
            position.y + dim.l > ship.dimensions.l):
            return False

    return True

def is_overweight(vehicles: Sequence[Vehicle], ship: Ship) -> bool:
    """Mengecek apakah total berat kendaraan melebihi kapasitas kapal"""
    total_weight = sum(vehicle.weight for vehicle in vehicles)

    if total_weight > ship.max_capacity:
        return True

    return False

def is_valid_state(vehicles: Sequence[Vehicle], positions: Sequence[Position | None], ship: Ship) -> bool:
    """Mengecek apakah susunan kendaraan valid."""
    if len(vehicles) != len(positions):
        return False

    ids = [vehicle.id for vehicle in vehicles]

    if len(ids) != len(set(ids)):
        return False

    for position in positions:
        if position is not None:
            if type(position.x) is not int or type(position.y) is not int:
                return False

    loaded_vehicles = []

    for i in range(len(vehicles)):
        if positions[i] is None:
            continue
```

```
            loaded_vehicles.append(vehicles[i])

            if not is_in_ship(vehicles[i], positions[i], ship):
                return False

            for j in range(i + 1, len(vehicles)):
                if positions[j] is None:
                    continue

                if is_overlapping(
                    vehicles[i], positions[i],
                    vehicles[j], positions[j],
                ):
                    return False

        if is_overweight(loaded_vehicles, ship):
            return False

        return True
```

1.3. Perhitungan Nilai Objektif

Fungsi calculate_objective menerima daftar kendaraan dan daftar posisi, lalu mengembalikan total tarif pengiriman. Fungsi menolak perbedaan panjang kedua daftar melalui ValueError. Setiap kendaraan yang mempunyai posisi selain None menyumbangkan shipping_fee ke nilai total.

Pemeriksaan geometri dan kapasitas dilakukan melalui is_valid_state. Pemisahan ini membuat perhitungan nilai dapat digunakan kembali setelah suatu individu dinyatakan valid.

Berikut source code calculate_objective pada src/init/objective.py

```
from collections.abc import Sequence
from .models import Vehicle, Position

def calculate_objective(vehicles: Sequence[Vehicle], positions:
```

```
    Sequence[Position | None]) -> float:
        """Menghitung nilai objektif dari susunan kendaraan"""
        if len(vehicles) != len(positions):
            raise ValueError("Jumlah kendaraan dan posisi harus sama")

        total_shipping_fee = 0.0

        for i in range(len(vehicles)):
            if positions[i] is not None:
                total_shipping_fee += vehicles[i].shipping_fee

        return total_shipping_fee
```

1.4. Pembentukan Initial State

Fungsi generate_initial_state memulai solusi dengan seluruh kendaraan berada di luar kapal. Fungsi kemudian mengacak urutan indeks kendaraan yang akan dicoba. Pengacakan urutan pemeriksaan ini tidak mengubah urutan daftar vehicles maupun hubungan indeks kendaraan dengan positions.

Untuk setiap kendaraan, fungsi mencoba koordinat dan orientasi random paling banyak max_attempts kali. Nilai bawaan max_attempts adalah 50. Kandidat diterima apabila keseluruhan state valid. Apabila kandidat tidak valid, posisi kendaraan dikembalikan ke None sebelum percobaan berikutnya. Kendaraan yang tidak berhasil ditempatkan tetap berada di luar kapal.

Koordinat dicoba dari rentang geladak dan baru diperiksa terhadap ukuran kendaraan melalui validator. Karena itu, sebagian kandidat dapat ditolak karena melewati batas, overlap, atau kelebihan kapasitas. Batas jumlah percobaan menjaga proses inisialisasi tetap berhingga

Berikut source code generate_initial_state pada src/init/initialization.py

```
import random
from collections.abc import Sequence
from .models import Vehicle, Position, Ship, Orientation
```

```
from .constraints import is_valid_state


def generate_initial_state(vehicles: Sequence[Vehicle], ship: Ship, rng: random.Random, max_attempts: int = 50) -> list[Position | None]:
    """Membuat susunan awal acak yang valid"""
    positions = [None] * len(vehicles)

    indices = list(range(len(vehicles)))
    rng.shuffle(indices)

    for i in indices:
        for _ in range(max_attempts):
            position = Position(
                x=rng.randrange(ship.dimensions.w),
                y=rng.randrange(ship.dimensions.l),
                orientation=rng.choice(list(Orientation)),
            )

            positions[i] = position

            if is_valid_state(vehicles, positions, ship):
                break

            positions[i] = None

    return positions
```

1.5. Operasi Neighbor

Modul neighbors.py menyediakan move, rotate, dan swap. Ketiganya membuat daftar posisi baru sehingga daftar solusi asal tetap tersedia.

- **Move**

  Berikut source code move pada src/init/neighbors.py

```
from collections.abc import Sequence
from .models import Position, Orientation
```

```
def move(positions: Sequence[Position | None], index: int, new_position: Position | None) -> list[Position | None]:
        """Memindahkan, memasukkan, atau mengeluarkan kendaraan"""
    new_positions = list(positions)
    new_positions[index] = new_position

    return new_positions
```

- **Rotate**

  Berikut source code move pada src/init/neighbors.py

```
def rotate(positions: Sequence[Position | None], index: int) -> list[Position | None]:
    """Merotasi kendaraan tanpa mengubah koordinatnya"""
    new_positions = list(positions)
    position = positions[index]

    if position is None:
        return new_positions

    if position.orientation == Orientation.HORIZONTAL:
        orientation = Orientation.VERTICAL
    else:
        orientation = Orientation.HORIZONTAL

    new_positions[index] = Position(
        x=position.x,
        y=position.y,
        orientation=orientation,
    )

    return new_positions
```

- **Swap**

  Berikut source code move pada src/init/neighbors.py

```
def swap(positions: Sequence[Position | None], index_a: int, index_b: int) -> list[Position | None]:
        """Menukar penempatan dua kendaraan, termasuk orientasinya"""
    new_positions = list(positions)

    new_positions[index_a], new_positions[index_b] = (
        new_positions[index_b],
        new_positions[index_a],
    )

    return new_positions
```

Rotasi kendaraan yang berada di luar kapal menghasilkan salinan state dengan nilai yang sama. Pertukaran dua penempatan juga dapat menghasilkan state yang tidak berubah apabila keduanya None. Operasi neighbor menghasilkan kandidat penempatan. Validitas kandidat tetap diperiksa oleh algoritma sebelum kandidat digunakan sebagai individu yang valid.

2. Implementasi dan Evaluasi Hill-Climbing with Sideways Move

Algoritma **Hill-Climbing with Sideways Move** digunakan untuk mencari konfigurasi penempatan kendaraan yang memaksimalkan total *shipping fee* pada geladak kapal. Pencarian dimulai dari suatu state awal yang valid, kemudian mengevaluasi seluruh tetangga yang dibangkitkan melalui operasi perpindahan, rotasi, dan pertukaran kendaraan.

Pada setiap iterasi, algoritma memilih tetangga dengan nilai objective tertinggi. Perpindahan diterima apabila meningkatkan nilai objective atau memiliki nilai yang sama selama batas *sideways move* belum tercapai. Apabila terdapat beberapa tetangga terbaik dengan nilai sama, salah satunya dipilih secara acak.

Implementasi terdiri atas dua fungsi utama, yaitu get_neighbors() untuk membangkitkan alternatif susunan dan hill_climbing() untuk menjalankan pencarian.

2.1. Menghasilkan neighbour: get_neighbors()

Fungsi ini menerima daftar kendaraan, susunan kendaraan saat ini, dan informasi kapal. Susunan direpresentasikan sebagai list positions, dengan setiap indeks berhubungan dengan kendaraan pada indeks yang sama dalam vehicles. Nilai None menunjukkan kendaraan yang tidak dimuat.

```
def get_neighbors(vehicles, positions, ship):
```

Setiap kandidat dibentuk dari state saat ini melalui satu operasi. Kandidat hanya ditambahkan sebagai tetangga apabila berbeda dari state semula dan memenuhi seluruh batasan:

```
if candidate != positions and is_valid_state(
    vehicles, candidate, ship
):
    neighbors.append(candidate)
```

Pemeriksaan validitas mencakup tidak adanya kendaraan bertumpuk, seluruh kendaraan yang dimuat berada dalam batas geladak, dan total berat tidak melebihi kapasitas kapal.

Pembuatan tetangga dilakukan menggunakan tiga operasi berikut.

1. Move

   Operasi ini mencoba memindahkan sebuah kendaraan ke seluruh koordinat integer yang memungkinkan kendaraan berada sepenuhnya di dalam kapal. Untuk kendaraan di luar kapal, kedua orientasi dicoba. Khusus kendaraan persegi, cukup digunakan satu orientasi karena kedua orientasi menghasilkan ukuran yang sama. Kendaraan yang sudah berada di kapal dipindahkan dengan mempertahankan orientasinya.

Batas koordinat penempatan dihitung berdasarkan ukuran geladak dan ukuran kendaraan setelah memperhitungkan orientasi:

```
max_x = ship.dimensions.w - dimensions.w
max_y = ship.dimensions.l - dimensions.l
```

Apabila salah satu batas bernilai negatif, kendaraan tidak dapat dimuat dengan orientasi tersebut. Selanjutnya, seluruh koordinat dalam batas tersebut diperiksa:

```
for x in range(max_x + 1):
   for y in range(max_y + 1):
      new_position = Position(x, y, orientation)
      candidate = move(positions, i, new_position)
```

Batas koordinat memastikan kendaraan berada dalam geladak. Pemeriksaan terhadap kendaraan lain dan kapasitas tetap dilakukan melalui is_valid_state().

2. Rotate

Operasi rotasi mengubah orientasi kendaraan dengan mempertahankan koordinat acuannya. Operasi ini hanya dilakukan terhadap kendaraan di kapal yang berbentuk bukan persegi.

```
if position is not None and vehicle.dimensions.w != vehicle.dimensions.l:
   candidate = rotate(positions, i)
```

Rotasi tidak langsung mengubah total *shipping fee*. Namun, rotasi dapat menghasilkan penataan ruang yang memungkinkan kendaraan lain dimasukkan pada iterasi berikutnya. Perpindahan tersebut dapat diterima sebagai *sideways move*.

3. Swap

   Operasi pertukaran dicoba untuk setiap pasangan kendaraan sebanyak satu kali.

```
for i in range(len(vehicles)):
    for j in range(i + 1, len(vehicles)):
        candidate = swap(positions, i, j)
```

   Pertukaran dapat melibatkan dua kendaraan di kapal maupun kendaraan di kapal dengan kendaraan di luar. Pertukaran antara kendaraan di dalam dan di luar memungkinkan penggantian kendaraan dengan kendaraan lain yang memberikan keuntungan lebih tinggi, selama susunan hasilnya valid. Kandidat yang tidak mengubah state tidak dimasukkan ke daftar tetangga.

2.2. Inisialisasi Pencarian: hill_climbing()

```
def hill_climbing(vehicles, positions, ship, rng, max_sideways=100):
```

Parameter:

- vehicles: Daftar kendaraan beserta atributnya
- positions: State awal yang valid
- ship: Dimensi geladak dan kapasitas kapal
- nrg: Pembangkit angka acak untuk memilih di antara kandidat terbaik yang nilainya sama
- max_sideways: Batas maksimum sideways berturut-turut

Pada awal pencarian, state awal disalin ke initial_positions untuk disimpan sebagai hasil eksperimen dan ke current sebagai state yang akan diperbarui. Nilai objective awal dihitung menggunakan calculate_objective().

```
initial_positions = list(positions)
current = list(positions)
current_value = calculate_objective(vehicles, current)

sideways_count = 0
total_sideways = 0
iterations = 0
history = [current_value]
```

sideways_count mencatat sideways berturut-turut, sedangkan total_sideways mencatat keseluruhan sideways selama pencarian. Variabel iterations menghitung perpindahan yang diterima. List history menyimpan perkembangan objective, dimulai dari state awal pada iterasi 0.

2.3. Pemilihan Tetangga Terbaik

Pada setiap iterasi, algoritma membangkitkan tetangga dari current. Apabila tidak terdapat tetangga valid, pencarian dihentikan. Selanjutnya, seluruh tetangga dievaluasi untuk memperoleh nilai objective tertinggi

```
best_value = calculate_objective(vehicles, neighbors[0])
best_neighbors = [neighbors[0]]

for candidate in neighbors[1:]:
    value = calculate_objective(vehicles, candidate)

    if value > best_value:
        best_value = value
        best_neighbors = [candidate]

    elif value == best_value:
        best_neighbors.append(candidate)
```

Ketika ditemukan nilai yang lebih tinggi, daftar kandidat terbaik diganti. Apabila ditemukan nilai yang sama, kandidat ditambahkan ke daftar.

Dengan demikian, best_neighbors berisi seluruh kandidat dengan objective tertinggi di antara tetangga yang dibangkitkan.

2.4. Penerimaan Perpindahan dan Sideways Move

Keputusan perpindahan ditentukan dengan membandingkan objective tetangga terbaik terhadap objective state saat ini.

1. **Nilai lebih rendah:** pencarian dihentikan karena algoritma tidak menerima penurunan objective.
2. **Nilai sama:** perpindahan diperbolehkan apabila batas sideways berturut-turut belum tercapai.
3. **Nilai lebih tinggi:** perpindahan diterima dan penghitung sideways berturut-turut direset menjadi nol.

```
if best_value < current_value:
    stop_reason = "local_optimum"
    break

if best_value == current_value:
    if sideways_count >= max_sideways:
        stop_reason = "sideways_limit"
        break

    sideways_count += 1
    total_sideways += 1

else:
    sideways_count = 0
```

Salah satu kandidat terbaik kemudian dipilih secara acak untuk menjadi state saat ini:

```
current = rng.choice(best_neighbors)
current_value = best_value

iterations += 1
history.append(current_value)
```

Implementasi juga menghentikan pencarian segera setelah perpindahan sideways yang diterima mencapai batas:

```
if sideways_count > 0 and sideways_count >= max_sideways:
    stop_reason = "sideways_limit"
    break
```

Batas sideways digunakan untuk membatasi perpindahan pada area dengan objective sama dan mencegah pencarian berulang tanpa peningkatan. Karena penghitung direset ketika terjadi peningkatan, total seluruh sideways dapat melebihi nilai max_sideways.

2.5. Kondisi Berhenti dan Hasil Pencarian

Algoritma berhenti ketika tidak terdapat tetangga valid, seluruh tetangga yang dibangkitkan memiliki objective lebih rendah, atau batas sideways berturut-turut tercapai.
Hasil yang dikembalikan mencakup:

Data	**Keterangan**
initial_state	Susunan kendaraan sebelum pencarian
final_state	Susunan kendaraan saat pencarian berhenti
final_objective	Total shipping fee pada state akhir
history	Riwayat objective sejak iterasi 0
iterations	Jumlah perpindahan yang diterima
total_sideways	Total perpindahan dengan objective sama
duration	Durasi pencarian
parameters	Parameter maksimum sideways

stop_reason	Alasan pencarian berhenti

Durasi diukur menggunakan perf_counter() sejak fungsi pencarian dimulai, sehingga tidak mencakup pembuatan initial state yang dilakukan sebelumnya.

Melalui mekanisme ini, nilai objective tidak menurun selama pencarian. Sideways memberi kesempatan untuk mengubah penataan kendaraan tanpa mengurangi keuntungan, sehingga dapat membuka peluang peningkatan berikutnya. Meskipun demikian, algoritma tetap dapat berhenti pada optimum lokal atau batas sideways dan tidak menjamin diperolehnya optimum global.

2.6. Keseluruhan Kode

```
from src.init.models import Position, Orientation
from src.init.operations import move, rotate, swap
from src.init.constraints import direction_of_car, is_valid_state
from time import perf_counter
from src.init.objective import calculate_objective

def get_neighbors(vehicles, positions, ship):
    """Menghasilkan daftar susunan tetangga yang berubah dan valid."""

    positions = list(positions)
    neighbors = []

    for i in range(len(vehicles)):
        vehicle = vehicles[i]
        position = positions[i]

        #Kendaraan di luar dapat dimasukkan dengan kedua orientasi
        #Kendaraan di kapal dipindahkan dengan orientasi yang sama
        if position is None:
            orientations = list(Orientation)
```

```
            #Untuk persegi, kedua orientasi memiliki bentuk yang
sama
          if vehicle.dimensions.w == vehicle.dimensions.l:
             orientations = [Orientation.HORIZONTAL]
       else:
          orientations = [position.orientation]

         #MOVE: mencoba semua koordinat yang muat di dalam
kapal
       for orientation in orientations:
          sample_position = Position(0, 0, orientation)
                              dimensions  =  direction_of_car(vehicle,
sample_position)

          #Batas koordinat agar kendaraan tetap dalam kapal
          max_x = ship.dimensions.w - dimensions.w
          max_y = ship.dimensions.l - dimensions.l

          if max_x < 0 or max_y < 0:
             continue

          for x in range(max_x + 1):
             for y in range(max_y + 1):
                new_position = Position(x, y, orientation)
                candidate = move(positions, i, new_position)

                                        if candidate != positions and
is_valid_state(vehicles, candidate, ship):
                    neighbors.append(candidate)

       #ROTATE: hanya kendaraan di kapal dan bukan persegi
             if position is not None and vehicle.dimensions.w !=
vehicle.dimensions.l:
          candidate = rotate(positions, i)

             if candidate != positions and is_valid_state(vehicles,
candidate, ship):
```

```
            neighbors.append(candidate)

    #SWAP: coba setiap pasangan kendaraan sekali
    for i in range(len(vehicles)):
        for j in range(i + 1, len(vehicles)):
            candidate = swap(positions, i, j)

                if candidate != positions and is_valid_state(vehicles,
candidate, ship):
                neighbors.append(candidate)

    return neighbors

def hill_climbing(vehicles, positions, ship, rng,
max_sideways=100):
        """Hill-Climbing with Sideways Move; batas sideways
berturut-turut."""
    start_time = perf_counter()

    initial_positions = list(positions)
    current = list(positions)
    current_value = calculate_objective(vehicles, current)

    sideways_count = 0
    total_sideways = 0
    iterations = 0
    history = [current_value]  #Nilai awal pada iterasi 0

    while True:
        neighbors = get_neighbors(vehicles, current, ship)

        if len(neighbors) == 0:
            stop_reason = "no_valid_neighbor"
            break

        #Cari semua kandidat dengan nilai objective tertinggi
        best_value = calculate_objective(vehicles, neighbors[0])
        best_neighbors = [neighbors[0]]
```

```
    for candidate in neighbors[1:]:
        value = calculate_objective(vehicles, candidate)

        if value > best_value:
            best_value = value
            best_neighbors = [candidate]

        elif value == best_value:
            best_neighbors.append(candidate)

    #Hill climbing tidak menerima penurunan nilai
    if best_value < current_value:
        stop_reason = "local_optimum"
        break

    if best_value == current_value:
        #Kandidat sama nilainya: gunakan jatah sideways
        if sideways_count >= max_sideways:
            stop_reason = "sideways_limit"
            break

        sideways_count += 1
        total_sideways += 1

    else:
                    #Ada peningkatan: reset hitungan sideways berturut-turut
        sideways_count = 0

       # Pilih acak jika beberapa kandidat memiliki nilai terbaik sama.
    current = rng.choice(best_neighbors)
    current_value = best_value

    iterations += 1
    history.append(current_value)
```

```
    #Hentikan setelah batas sideways tercapai
            if sideways_count > 0 and sideways_count >=
max_sideways:
        stop_reason = "sideways_limit"
        break

  duration = perf_counter() - start_time

  return {
    "initial_state": initial_positions,
    "final_state": current,
    "final_objective": current_value,
    "history": history,
    "iterations": iterations,
    "total_sideways": total_sideways,
    "duration": duration,
    "parameters": {"max_sideways": max_sideways},
    "stop_reason": stop_reason,
  }
```

3. Implementasi dan Evaluasi Simulated Annealing

Algoritma Simulated Annealing digunakan untuk mencari konfigurasi penempatan kendaraan yang memaksimalkan total tarif pengiriman. Pencarian dimulai dari suatu state awal, kemudian mengevaluasi kandidat tetangga. Implementasi utama dari pencarian ini terdiri atas dua fungsi utama, yaitu pengecekan optimum lokal dan pengulangan dari algoritma itu sendiri.

3.1. Inisialisasi State Awal

Algoritma dimulai dengan menentukan *state* awal melalui fungsi generate_initial_state (jika state awal tidak diberikan). State yang dihasilkan harus dipastikan terlebih dahulu apakah sudah memenuhi constraints yang berlaku. Kemudian, dilakukan perhitungan nilai objektif dan state awal disimpan menjadi solusi sementara yang akan dibandingkan dengan kandidat solusi berikutnya. Berikut source code untuk inisialisasi state awal.

```
rng = random.Random(seed)
start_time = perf_counter()

current = (
    generate_initial_state(vehicles, ship, rng)
    if initial is None else list(initial)
)

if not is_valid_state(vehicles, current, ship):
    raise ValueError("state awal tidak valid")

initial_state = list(current)
current_value = calculate_objective(vehicles, current)
best, best_value = list(current), current_value
```

rng digunakan untuk menghasilkan pilihan acak selama berjalannya satu run dengan seed untuk melakukan pengacakan urutan ulang kondisi awal dan konfigurasi yang sama.

3.2. Pencarian Neighbor dan Local Optimum

Pada setiap iterasi, algoritma memilih salah satu operasi, yaitu *move, rotate,* atau *swap* untuk menghasilkan kandidat neighbor. Kandidat yang digunakan harus berbeda dengan state saat ini. Pencarian kandidat dilakukan maksimal 100 kali dalam satu iterasi. Berikut source code untuk pemilihan operasi dan validasi kandidat.

```
candidate = None

for _ in range(100):
    i = rng.randrange(len(vehicles))
    action = rng.choice(["move", "rotate", "swap"])

    # Kandidat dihasilkan menggunakan operasi yang dipilih.

    if neighbor != current and is_valid_state(
        vehicles, neighbor, ship
    ):
        candidate = neighbor
        break
```

Terdapat pula fungsi is_local_optimum untuk memeriksa apakah terdapat neighbor valid yang memiliki nilai objective lebih tinggi. Jika tidak ditemukan tetangga yang lebih baik, state dianggap sebagai local optimum. Frekuensi local optimum dihitung ketika algoritma memasuki kondisi tersebut, termasuk apabila state awal sudah merupakan local optimum. Kondisi yang bertahan tanpa perubahan tidak menambah hitungan. Berikut source code untuk pencatatan local optimum.

```
is_local_optimum = (
    bool(vehicles)
    and is_local_optimum(vehicles, current, ship)
)
local_optimum_count = int(in_local_optimum)
local_optimum_history = [in_local_optimum]
```

3.3. Evaluasi Kandidat dan Penerimaan Solusi

Setiap kandidat neighbor yang valid dievaluasi menggunakan calculate_objective. Selisih antara objective kandidat dan objective saat ini disimpan dalam variabel delta. Kandidat dengan objective lebih tinggi atau sama langsung diterima dan kandidat dengan objective lebih rendah dapat diterima berdasarkan probabilitas yang dipengaruhi oleh temperatur.

Berikut source code untuk evaluasi dan penerimaan kandidat.

```
new_value = calculate_objective(vehicles, candidate)
delta = new_value - current_value

probability = (
   1.0 if delta >= 0
   else math.exp(delta / temperature)
)

if delta >= 0 or rng.random() < probability:
   current = candidate
   current_value = new_value

   if current_value > best_value:
      best, best_value = list(current), current_value
```

3.4. Pencatatan Hasil dan Penghentian Algoritma

Selama proses pencarian, algoritma mencatat riwayat objective, probabilitas penerimaan, suhu, perubahan objective, serta kondisi local optimum. Suhu dihitung berdasarkan suhu awal dan faktor penurunan alpha. Proses dihentikan ketika suhu turun di bawah suhu minimum, jumlah iterasi mencapai batas, atau tidak terdapat kendaraan untuk

diproses. Berikut source code untuk pencatatan suhu dan perubahan objective.

```
temperature = T0 * alpha ** iteration

if temperature < T_min:
   stop_reason = "temperature_limit"
   break

temperatures.append(temperature)

new_value = calculate_objective(vehicles, candidate)
delta = new_value - current_value

deltas.append(delta)
probs.append(probability)
history.append(current_value)
```

3.5. Keseluruhan Kode

```
def is_local_optimum(vehicles, positions, ship):
   """True jika tidak ada satu operasi valid yang meningkatkan
objective."""
   # Memindahkan atau merotasi kendaraan di kapal tidak
mengubah shipping fee.
   # Periksa move yang bisa menaikkan nilai: memasukkan atau
mengeluarkan mobil.
   for i, vehicle in enumerate(vehicles):
      if positions[i] is not None:
         # Pengeluaran hanya meningkatkan objective jika tarifnya
negatif.
         if vehicle.shipping_fee < 0:
            candidate = move(positions, i, None)
            if is_valid_state(vehicles, candidate, ship):
               return False
         continue

      if vehicle.shipping_fee <= 0:
```

```
            continue

        orientations = list(Orientation)
        if vehicle.dimensions.w == vehicle.dimensions.l:
            orientations = [Orientation.HORIZONTAL]

        for orientation in orientations:
            dimensions = direction_of_car(vehicle, Position(0, 0,
orientation))
            max_x = ship.dimensions.w - dimensions.w
            max_y = ship.dimensions.l - dimensions.l

            for x in range(max_x + 1):
                for y in range(max_y + 1):
                    candidate = move(positions, i, Position(x, y, orientation))
                    if is_valid_state(vehicles, candidate, ship):
                        return False

    # Swap dapat menaikkan nilai jika kendaraan dari luar lebih
menguntungkan.
    for i in range(len(vehicles)):
        for j in range(i + 1, len(vehicles)):
            if (positions[i] is None) == (positions[j] is None):
                continue

            outside = i if positions[i] is None else j
            inside = j if positions[i] is None else i
            if vehicles[outside].shipping_fee <=
vehicles[inside].shipping_fee:
                continue

            candidate = swap(positions, i, j)
            if is_valid_state(vehicles, candidate, ship):
                return False

    return True
```

```
def simulated_annealing(
    vehicles, ship, T0: float = 100, alpha=0.999, T_min=0.1,
    max_iter=5000, initial=None, seed=11,
):
    """Menjalankan SA serta mencatat objective, probabilitas, dan local
optimum."""
    if T0 <= 0 or not 0 < alpha < 1 or T_min <= 0:
        raise ValueError("suhu tidak valid")
    if type(max_iter) is not int or max_iter < 0:
        raise ValueError("max_iter harus integer >= 0.")

    rng = random.Random(seed)
    start_time = perf_counter()

    current = (
        generate_initial_state(vehicles, ship, rng)
        if initial is None else list(initial)
    )

    if not is_valid_state(vehicles, current, ship):
        raise ValueError("state awal tidak valid")

    initial_state = list(current)
    current_value = calculate_objective(vehicles, current)
    best, best_value = list(current), current_value
    history, probs = [current_value], []
    temperatures = []  # Suhu untuk setiap iterasi, mulai iterasi 1.
    deltas = []        # Objective kandidat dikurangi objective sebelum
keputusan.

    # State awal ikut dihitung jika sudah berada pada local optimum.
    in_local_optimum = bool(vehicles) and
is_local_optimum(vehicles, current, ship)
    local_optimum_count = int(in_local_optimum)
    local_optimum_history = [in_local_optimum]  # Sejajar dengan
history.

    rejected_moves = 0
```

```
worse_accepted = 0
failed_neighbor = 0
iterations = 0
stop_reason = "iteration_limit" if vehicles else "no_vehicles"

for iteration in range(1, max_iter + 1):
    if not vehicles:
        break

    temperature = T0 * alpha ** iteration
    if temperature < T_min:
        stop_reason = "temperature_limit"
        break

    temperatures.append(temperature)
    candidate = None

    for _ in range(100):
        i = rng.randrange(len(vehicles))
        action = rng.choice(["move", "rotate", "swap"])

        if action == "move":
            if current[i] is None:
                position = Position(
                    rng.randrange(ship.dimensions.w),
                    rng.randrange(ship.dimensions.l),
                    rng.choice(list(Orientation)),
                )
                neighbor = move(current, i, position)
            elif rng.random() < 0.15:
                neighbor = move(current, i, None)
            else:
                position = Position(
                    rng.randrange(ship.dimensions.w),
                    rng.randrange(ship.dimensions.l),
                    current[i].orientation,
                )
                neighbor = move(current, i, position)
```

```
        elif action == "rotate":
            neighbor = rotate(current, i)

        elif len(vehicles) > 1:
            j = rng.randrange(len(vehicles) - 1)
            if j >= i:
                j += 1
            neighbor = swap(current, i, j)

        else:
            continue

        if neighbor != current and is_valid_state(
            vehicles, neighbor, ship
        ):
            candidate = neighbor
            break

    if candidate is None:
        failed_neighbor += 1
        probs.append(None)
        deltas.append(None)  # Tidak ada kandidat untuk dihitung.
        history.append(current_value)
        local_optimum_history.append(in_local_optimum)
        iterations = iteration
        continue

    new_value = calculate_objective(vehicles, candidate)
    delta = new_value - current_value
    deltas.append(delta)

    probability = 1.0 if delta >= 0 else math.exp(delta / temperature)
    probs.append(probability)

    if delta >= 0 or rng.random() < probability:
        current = candidate
        current_value = new_value
```

```
            if delta < 0:
                worse_accepted += 1

            if current_value > best_value:
                best, best_value = list(current), current_value

            # Periksa ulang hanya setelah state berubah.
            was_local_optimum = in_local_optimum
            in_local_optimum = is_local_optimum(vehicles, current, ship)
            if in_local_optimum and not was_local_optimum:
                local_optimum_count += 1
        else:
            rejected_moves += 1

        history.append(current_value)
        local_optimum_history.append(in_local_optimum)
        iterations = iteration

    duration = perf_counter() - start_time

    return {
        "initial_state": initial_state,
        "initial_objective": calculate_objective(vehicles, initial_state),
        "final_state": list(current),
        "final_objective": current_value,
        "best_state": best,
        "best_objective": best_value,
        "history": history,
        "probs": probs,
        "temperatures": temperatures,
        "deltas": deltas,
        "local_optimum_count": local_optimum_count,
        "local_optimum_history": local_optimum_history,
        "local_optimum_definition": (
            "Tidak ada satu move, rotate, atau swap valid dengan objective
"
            "lebih tinggi. Hitung saat masuk kondisi ini, termasuk state awal;
```

```
"
        "bertahan pada kondisi yang sama tidak menambah
hitungan."
    ),
    "iterations": iterations,
    "duration": duration,
    "stop_reason": stop_reason,
    "seed": seed,
    "parameters": {
      "T0": T0,
      "alpha": alpha,
      "T_min": T_min,
      "max_iter": max_iter,
      "seed": seed,
    },
    "failed_neighbor": failed_neighbor,
    "rejected_moves": rejected_moves,
    "worse_accepted": worse_accepted,
  }
```

4. Implementasi dan Evaluasi Genetic Algorithm

Genetic Algorithm mencari solusi melalui populasi yang berisi beberapa susunan kendaraan. Pada setiap generasi, individu dipilih sebagai induk berdasarkan fitness, kemudian digabungkan melalui crossover dan dapat mengalami mutasi. Implementasi utama berada pada src/algorithms/genetic_algorithm.py.

4.1. Representasi Individu dan Populasi

Satu individu merupakan daftar Position atau None. Setiap elemen daftar berperan sebagai gen yang menyatakan penempatan satu kendaraan pada indeks yang bersesuaian. Seluruh individu mempunyai jumlah gen yang sama dengan jumlah kendaraan. Populasi merupakan daftar yang berisi sejumlah individu tersebut.

Kromosom menyimpan penempatan kendaraan, bukan urutan objek Vehicle. Dengan demikian, crossover menggabungkan penempatan pada indeks yang sama dan tetap mempertahankan identitas kendaraan. Perubahan kombinasi penempatan tetap dapat menimbulkan overlap atau kelebihan kapasitas, sehingga offspring perlu diperiksa kembali.

4.2. Inisialisasi Populasi

Fungsi generate_genome membuat satu individu dengan memanggil generate_initial_state. Fungsi generate_population memanggil generate_genome sebanyak population_size kali. Seluruh pemanggilan menerima objek rng yang sama untuk satu run. State generator random berkembang pada setiap pemanggilan sehingga proses pembentukan individu tidak dimulai ulang dengan seed yang sama.

Individu awal dihasilkan secara random dengan memenuhi batasan pada data persoalan yang valid. Pengacakan dapat menghasilkan susunan yang berbeda, tetapi implementasi tidak memaksakan seluruh individu mempunyai kromosom yang unik.

Berikut source code untuk pembentukan individual dan populasi

```
def generate_genome(vehicles, ship, rng):
    """Membuat satu individu berupa susunan kendaraan yang valid."""
    return generate_initial_state(vehicles, ship, rng)

def generate_population(vehicles, ship, population_size, rng):
    """Membuat kumpulan individu awal."""
    return [
        generate_genome(vehicles, ship, rng)
        for _ in range(population_size)
    ]
```

4.3. Evaluasi Fitness

Fungsi fitness meneruskan individu ke calculate_objective. Nilai fitness sama dengan total ShippingFee kendaraan yang diangkut. Individu dengan nilai lebih tinggi dianggap lebih baik dalam persoalan maksimisasi ini.

Berikut source code untuk fungsi fitness

```
def fitness(vehicles, individual):
    """Menghitung fitness individu yang sudah valid."""
    return calculate_objective(vehicles, individual)
```

Fitness digunakan untuk individu yang sudah valid. Pada alur utama, offspring yang tidak valid diganti terlebih dahulu sebelum fitness populasi berikutnya dihitung. Dengan mekanisme tersebut, implementasi tidak menggunakan pengurangan fitness berupa penalti pelanggaran constraints.

4.4. Seleksi Induk (Parent)

Fungsi select_parent menggunakan roulette-wheel selection. Pemilihan dilakukan dengan rng.choices dan fitness_values sebagai bobot.

Berikut source code untuk fungsi select_parent

```
def select_parent(population, fitness_values, rng):
    """Roulette wheel: fitness lebih tinggi memberi peluang lebih besar."""
    if max(fitness_values) == 0:
        return rng.choice(population)
    return rng.choices(population, weights=fitness_values, k=1)[0]
```

4.5. Crossover Satu Titik

Fungsi crossover menggunakan satu titik potong. Untuk kromosom dengan setidaknya dua gen, titik potong dipilih secara random dari

indeks 1 sampai panjang kromosom dikurangi 1. Child mengambil bagian sebelum titik potong dari parent_a dan bagian mulai titik potong dari parent_b.

Berikut source code untuk crossover

```
def crossover(parent_a, parent_b, rng):
    """Satu child dari satu titik potong; parent tidak diubah."""
    if len(parent_a) < 2:
        return list(parent_a)
    cut = rng.randrange(1, len(parent_a))
    return parent_a[:cut] + parent_b[cut:]
```

Sebagai ilustrasi, untuk empat kendaraan dan titik potong 2, child memperoleh penempatan kendaraan pertama dan kedua dari parent_a serta kendaraan ketiga dan keempat dari parent_b. Ilustrasi ini menunjukkan cara pewarisan gen, bukan hasil eksperimen. Jika jumlah gen kurang dari dua, fungsi mengembalikan salinan parent_a.

Setiap pembentukan child selalu menjalankan crossover. Implementasi tidak mempunyai parameter probabilitas crossover terpisah. Peluang tambahan yang dapat diatur pada alur tersebut adalah mutation_rate.

4.6. Mutasi Individu

Setelah crossover, fungsi utama membandingkan rng.random() dengan mutation_rate. Jika syarat terpenuhi, fungsi mutate mencoba satu operasi random pada child. Mutation rate berlaku pada satu child, bukan pada setiap gen secara terpisah.

Fungsi mutate memilih salah satu dari empat jenis mutasi dengan peluang yang sama. Move mencoba penempatan random baru dan dapat memasukkan kendaraan yang sebelumnya berada di luar kapal. Remove mengganti satu posisi menjadi None. Rotate mengubah orientasi kendaraan pada koordinat yang sama. Swap menukar dua

penempatan dengan indeks berbeda apabila tersedia setidaknya dua kendaraan.

Berikut source code untuk fungsi mutate

```
def mutate(individual, ship, rng):
    """Mencoba satu perubahan acak pada individu."""
    if not individual:
        return list(individual)

    index = rng.randrange(len(individual))
    mutation_type = rng.choice(["move", "remove", "rotate", "swap"])

    if mutation_type == "move":
        new_position = Position(
            x=rng.randrange(ship.dimensions.w),
            y=rng.randrange(ship.dimensions.l),
            orientation=rng.choice(list(Orientation)),
        )
        return move(individual, index, new_position)

    elif mutation_type == "remove":
        return move(individual, index, None)

    elif mutation_type == "rotate":
        return rotate(individual, index)

    elif len(individual) >= 2:
        index_a, index_b = rng.sample(range(len(individual)), 2)
        return swap(individual, index_a, index_b)

    return list(individual)
```

Operasi remove menggunakan fungsi move dengan posisi tujuan None, sehingga perubahan tersebut merupakan pengeluaran kendaraan dari kapal. Beberapa mutasi dapat menghasilkan state yang sama, misalnya rotasi kendaraan yang tidak diangkut atau penghapusan gen yang sudah None. Pemanggilan mutate menyatakan percobaan

perubahan, sehingga setiap pemanggilan tidak selalu mengubah nilai objektif maupun susunan.

4.7. Pembentukan Generasi dan Penyimpanan Solusi Terbaik

Fungsi run_genetic_algorithm menerima vehicles, ship, population_size, generations, mutation_rate, dan rng. Fungsi memeriksa bahwa ukuran populasi merupakan integer minimal 1, jumlah generasi merupakan integer minimal 0, dan mutation rate berada pada rentang 0 sampai 1.

Setelah populasi awal terbentuk, fungsi menghitung fitness seluruh individu. Individu terbaik pada populasi awal disalin ke best_individual dan nilainya disimpan sebagai best_objective. Salinan setiap individu awal juga disimpan pada initial_population. Iterasi 0 pada history merepresentasikan populasi awal tersebut.

Berikut source code untuk run_genetic_algorithm

```
def run_genetic_algorithm(
    vehicles,
    ship,
    population_size,
    generations,
    mutation_rate,
    rng,
):
    """Menjalankan GA dan mencatat hasil pencarian."""
    if type(population_size) is not int or population_size < 1:
        raise ValueError("Jumlah populasi harus integer minimal 1.")

    if type(generations) is not int or generations < 0:
        raise ValueError("Jumlah generasi harus integer minimal 0.")

    if not 0 <= mutation_rate <= 1:
        raise ValueError("Mutation rate harus antara 0 dan 1.")
```

```
# Durasi mencakup inisialisasi populasi dan seluruh generasi.
start = perf_counter()

population = generate_population(
    vehicles, ship, population_size, rng
)

# Position harus memakai frozen=True.
initial_population = [
    individual.copy() for individual in population
]

fitness_values = [
    fitness(vehicles, individual)
    for individual in population
]

# Simpan salinan solusi terbaik sejak populasi awal.
best_index = fitness_values.index(max(fitness_values))
best_individual = population[best_index].copy()
best_objective = fitness_values[best_index]

# Iterasi 0 adalah populasi awal.
history = [{
    "iteration": 0,
    "maximum": max(fitness_values),
    "average": sum(fitness_values) / population_size,
}]

for generation in range(1, generations + 1):
    new_population = []

    for _ in range(population_size):
        parent_a = select_parent(population, fitness_values, rng)
        parent_b = select_parent(population, fitness_values, rng)

        child = crossover(parent_a, parent_b, rng)
```

```
        if rng.random() < mutation_rate:
            child = mutate(child, ship, rng)

        if not is_valid_state(vehicles, child, ship):
            child = generate_genome(vehicles, ship, rng)

        new_population.append(child)

    population = new_population

    fitness_values = [
        fitness(vehicles, individual)
        for individual in population
    ]

    best_index = fitness_values.index(max(fitness_values))
    if fitness_values[best_index] > best_objective:
        best_individual = population[best_index].copy()
        best_objective = fitness_values[best_index]

    history.append({
        "iteration": generation,
        "maximum": max(fitness_values),
        "average": sum(fitness_values) / population_size,
    })

duration = perf_counter() - start

return {
    "initial_population": initial_population,
    "final_population": population,
    "best_individual": best_individual,
    "best_objective": best_objective,
    "history": history,
    "population_size": population_size,
    "iterations": generations,
    "mutation_rate": mutation_rate,
    "duration_seconds": duration,
```

```
}
```

4.8. Pencatatan dan Penyimpanan Hasil

Pada setiap generasi, history mencatat nomor iterasi, fitness maksimum, dan fitness rata-rata populasi aktif. Nilai rata-rata diperoleh dari jumlah fitness dibagi population_size. Catatan maksimum pada history berbeda makna dengan best_objective yang mencakup seluruh pencarian.

Durasi diukur menggunakan perf_counter. Pengukuran dimulai sebelum pembentukan populasi awal dan dihentikan setelah seluruh generasi serta riwayatnya selesai diproses. Oleh karena itu, duration_seconds mencakup inisialisasi populasi, evaluasi fitness, seleksi, crossover, mutasi, penggantian child tidak valid, dan pencatatan riwayat. Penulisan berkas serta pembuatan grafik dilakukan setelah fungsi utama mengembalikan hasil dan berada di luar pengukuran tersebut.

Kunci Hasil	**Isi**
initial_population	Salinan seluruh individu pada populasi awal
final_population	Populasi aktif setelah generasi terakhir
best_individual	Salinan solusi terbaik yang ditemukan sepanjang pencarian
best_objective	Nilai objektif solusi terbaik sepanjang pencarian
history	Iterasi serta nilai maksimum dan rata-rata setiap populasi
population_size	Jumlah individu pada setiap generasi
iterations	Jumlah generasi yang dijalankan
mutation_rate	Probabilitas percobaan mutasi pada setiap

	child
duration_seconds	Durasi proses pencarian dalam detik

4.9. Keseluruhan Kode

```
from ..init.constraints import is_valid_state
from ..init.objective import calculate_objective
from ..init.initialization import generate_initial_state
from ..init.models import Position, Orientation
from ..init.operations import move, rotate, swap
from time import perf_counter
import random

def generate_genome(vehicles, ship, rng):
    """Membuat satu individu berupa susunan kendaraan yang valid."""
    return generate_initial_state(vehicles, ship, rng)

def generate_population(vehicles, ship, population_size, rng):
    """Membuat kumpulan individu awal."""
    return [
        generate_genome(vehicles, ship, rng)
        for _ in range(population_size)
    ]

def fitness(vehicles, individual):
    """Menghitung fitness individu yang sudah valid."""
    return calculate_objective(vehicles, individual)

def select_parent(population, fitness_values, rng):
    """Roulette wheel: fitness lebih tinggi memberi peluang lebih besar."""
    if max(fitness_values) == 0:
        return rng.choice(population)
    return rng.choices(population, weights=fitness_values, k=1)[0]

def crossover(parent_a, parent_b, rng):
    """Satu child dari satu titik potong; parent tidak diubah."""
```

```
    if len(parent_a) < 2:
        return list(parent_a)
    cut = rng.randrange(1, len(parent_a))
    return parent_a[:cut] + parent_b[cut:]

def mutate(individual, ship, rng):
    """Mencoba satu perubahan acak pada individu."""
    if not individual:
        return list(individual)

    index = rng.randrange(len(individual))
    mutation_type = rng.choice(["move", "remove", "rotate", "swap"])

    if mutation_type == "move":
        new_position = Position(
            x=rng.randrange(ship.dimensions.w),
            y=rng.randrange(ship.dimensions.l),
            orientation=rng.choice(list(Orientation)),
        )
        return move(individual, index, new_position)

    elif mutation_type == "remove":
        return move(individual, index, None)

    elif mutation_type == "rotate":
        return rotate(individual, index)

    elif len(individual) >= 2:
        index_a, index_b = rng.sample(range(len(individual)), 2)
        return swap(individual, index_a, index_b)

    return list(individual)

def run_genetic_algorithm(
    vehicles,
    ship,
    population_size=20,
    generations=100,
```

```
    mutation_rate=0.1,
    rng=None,
):
    """Menjalankan GA; parameter yang tidak diberikan memakai nilai
bawaan."""
    if rng is None:
        rng = random.Random()

    if type(population_size) is not int or population_size < 1:
        raise ValueError("Jumlah populasi harus integer minimal 1.")

    if type(generations) is not int or generations < 0:
        raise ValueError("Jumlah generasi harus integer minimal 0.")

    if not 0 <= mutation_rate <= 1:
        raise ValueError("Mutation rate harus antara 0 dan 1.")

    # Durasi mencakup inisialisasi populasi dan seluruh generasi.
    start = perf_counter()

    population = generate_population(
        vehicles, ship, population_size, rng
    )

    # Position harus memakai frozen=True.
    initial_population = [
        individual.copy() for individual in population
    ]

    fitness_values = [
        fitness(vehicles, individual)
        for individual in population
    ]

    # Simpan salinan solusi terbaik sejak populasi awal.
    best_index = fitness_values.index(max(fitness_values))
    best_individual = population[best_index].copy()
    best_objective = fitness_values[best_index]
```

```
# Iterasi 0 adalah populasi awal.
history = [{
   "iteration": 0,
   "maximum": max(fitness_values),
   "average": sum(fitness_values) / population_size,
}]

for generation in range(1, generations + 1):
   new_population = []

   for _ in range(population_size):
      parent_a = select_parent(population, fitness_values, rng)
      parent_b = select_parent(population, fitness_values, rng)

      child = crossover(parent_a, parent_b, rng)

      if rng.random() < mutation_rate:
         child = mutate(child, ship, rng)

      if not is_valid_state(vehicles, child, ship):
         child = generate_genome(vehicles, ship, rng)

      new_population.append(child)

   population = new_population

   fitness_values = [
      fitness(vehicles, individual)
      for individual in population
   ]

   best_index = fitness_values.index(max(fitness_values))
   if fitness_values[best_index] > best_objective:
      best_individual = population[best_index].copy()
      best_objective = fitness_values[best_index]

   history.append({
```

```
        "iteration": generation,
        "maximum": max(fitness_values),
        "average": sum(fitness_values) / population_size,
    })

duration = perf_counter() - start

return {
    "initial_population": initial_population,
    "final_population": population,
    "best_individual": best_individual,
    "best_objective": best_objective,
    "history": history,
    "population_size": population_size,
    "iterations": generations,
    "mutation_rate": mutation_rate,
    "duration_seconds": duration,
    "duration_scope": "initialization_and_search",
}
```

5. Hasil dan Analisis Eksperimen

- **Hasil**

```
HC
```

```
Dataset: medium.json
Seeds: [42, 43, 44]
Sesi dibuat. Menyiapkan percobaan pertama.
Menjalankan 1/3 · HC_seed-42
Tersimpan 1/3 · HC_seed-42

HC — RUN 1 — SEED 42

STATE AWAL
V001: tidak dimuat
V002: tidak dimuat
V003: tidak dimuat
V004: tidak dimuat
V005: (1, 0), orientasi=horizontal
V006: (6, 1), orientasi=vertical
V007: tidak dimuat
V008: tidak dimuat
V009: tidak dimuat
V010: tidak dimuat
V011: (3, 1), orientasi=horizontal
V012: tidak dimuat
V013: tidak dimuat
V014: tidak dimuat
V015: (6, 4), orientasi=horizontal
V016: (2, 6), orientasi=vertical
V017: tidak dimuat
V018: tidak dimuat
V019: (0, 7), orientasi=horizontal
V020: tidak dimuat
V021: tidak dimuat
V022: (4, 8), orientasi=horizontal
V023: tidak dimuat
V024: (0, 3), orientasi=vertical
Objective: 1750.0
Total berat: 79 / 100
Valid: True

STATE AKHIR
V001: (3, 8), orientasi=horizontal
V002: tidak dimuat
V003: tidak dimuat
V004: (6, 0), orientasi=vertical
V005: (3, 3), orientasi=horizontal
V006: tidak dimuat
V007: tidak dimuat
V008: tidak dimuat
V009: tidak dimuat
V010: (1, 6), orientasi=vertical
V011: (0, 8), orientasi=horizontal
V012: tidak dimuat
V013: tidak dimuat
V014: tidak dimuat
V015: (2, 1), orientasi=horizontal
V016: tidak dimuat
V017: tidak dimuat
V018: tidak dimuat
V019: (0, 0), orientasi=horizontal
V020: (10, 3), orientasi=horizontal
V021: tidak dimuat
V022: (6, 3), orientasi=horizontal
V023: (9, 7), orientasi=vertical
V024: (0, 3), orientasi=vertical
Objective: 2720.0
Total berat: 94 / 100
Valid: True
Alasan berhenti: sideways_limit
Batas sideways: 100
Total sideways: 101
Objective hasil: 2720.0
Jumlah iterasi: 107
Durasi: 1.7267 detik
Menjalankan 2/3 · HC_seed-43
Tersimpan 2/3 · HC_seed-43
```

```
HC - RUN 2 - SEED 43

STATE AWAL
V001: (3, 0), orientasi=horizontal
V002: tidak dimuat
V003: (1, 2), orientasi=vertical
V004: tidak dimuat
V005: tidak dimuat
V006: tidak dimuat
V007: tidak dimuat
V008: (9, 0), orientasi=horizontal
V009: tidak dimuat
V010: tidak dimuat
V011: (1, 8), orientasi=vertical
V012: tidak dimuat
V013: tidak dimuat
V014: tidak dimuat
V015: tidak dimuat
V016: tidak dimuat
V017: tidak dimuat
V018: tidak dimuat
V019: (2, 6), orientasi=vertical
V020: (9, 4), orientasi=horizontal
V021: tidak dimuat
V022: (7, 5), orientasi=vertical
V023: (6, 2), orientasi=horizontal
V024: (4, 7), orientasi=vertical
Objective: 2150.0
Total berat: 86 / 100
Valid: True

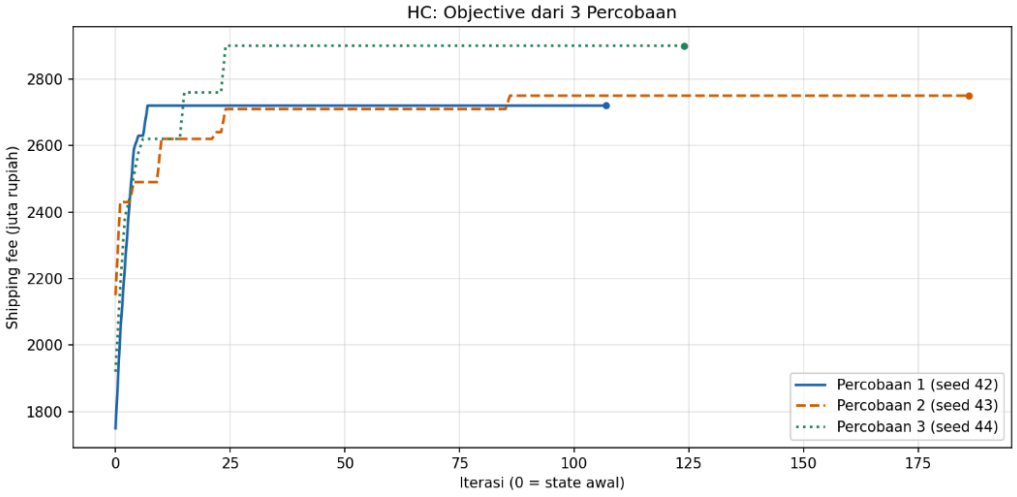
STATE AKHIR
V001: (2, 0), orientasi=horizontal
V002: tidak dimuat
V003: tidak dimuat
V004: tidak dimuat
V005: (6, 4), orientasi=vertical
V006: tidak dimuat
V007: tidak dimuat
V008: tidak dimuat
V009: tidak dimuat
V010: tidak dimuat
V011: (6, 2), orientasi=vertical
V012: tidak dimuat
V013: (9, 3), orientasi=horizontal
V014: tidak dimuat
V015: (7, 6), orientasi=vertical
V016: tidak dimuat
V017: tidak dimuat
V018: tidak dimuat
V019: (4, 2), orientasi=vertical
V020: (4, 4), orientasi=horizontal
V021: (0, 2), orientasi=vertical
V022: (3, 8), orientasi=horizontal
V023: (9, 0), orientasi=horizontal
V024: (0, 7), orientasi=horizontal
Objective: 2750.0
Total berat: 95 / 100
Valid: True
Alasan berhenti: sideways_limit
Batas sideways: 100
Total sideways: 180
Objective hasil: 2750.0
Jumlah iterasi: 186
Durasi: 3.2260 detik
Menjalankan 3/3 · HC_seed-44
Tersimpan 3/3 · HC_seed-44
```

```
HC — RUN 3 — SEED 44

STATE AWAL
V001: tidak dimuat
V002: tidak dimuat
V003: tidak dimuat
V004: tidak dimuat
V005: (0, 0), orientasi=horizontal
V006: tidak dimuat
V007: (5, 5), orientasi=vertical
V008: tidak dimuat
V009: tidak dimuat
V010: tidak dimuat
V011: (1, 8), orientasi=horizontal
V012: (5, 1), orientasi=vertical
V013: tidak dimuat
V014: tidak dimuat
V015: (7, 7), orientasi=horizontal
V016: tidak dimuat
V017: tidak dimuat
V018: tidak dimuat
V019: (9, 1), orientasi=vertical
V020: (2, 0), orientasi=horizontal
V021: tidak dimuat
V022: (0, 6), orientasi=horizontal
V023: (4, 7), orientasi=horizontal
V024: tidak dimuat
Objective: 1920.0
Total berat: 78 / 100
Valid: True

STATE AKHIR
V001: (7, 8), orientasi=horizontal
V002: tidak dimuat
V003: tidak dimuat
V004: tidak dimuat
V005: (4, 0), orientasi=horizontal
V006: tidak dimuat
V007: (4, 3), orientasi=vertical
V008: tidak dimuat
V009: (2, 0), orientasi=horizontal
V010: (4, 5), orientasi=vertical
V011: (2, 8), orientasi=horizontal
V012: tidak dimuat
V013: tidak dimuat
V014: tidak dimuat
V015: tidak dimuat
V016: tidak dimuat
V017: tidak dimuat
V018: tidak dimuat
V019: (0, 0), orientasi=vertical
V020: (0, 2), orientasi=horizontal
V021: tidak dimuat
V022: (0, 6), orientasi=vertical
V023: (9, 0), orientasi=vertical
V024: (4, 7), orientasi=horizontal
Objective: 2900.0
Total berat: 100 / 100
Valid: True
Alasan berhenti: sideways_limit
Batas sideways: 100
Total sideways: 116
Objective hasil: 2900.0
Jumlah iterasi: 124
Durasi: 2.0689 detik
Selesai. 3 percobaan sudah tersimpan.

HC: 3/3 run tersimpan.
Folder: /Users/acit/orca/workspaces/IF3070_G22_pararunten/daniel/output/hill_climbing/hc_20261022_210045_370694
Sesi dibuat. Menyiapkan percobaan pertama.
Menjalankan 1/3 · SA_seed-42
Tersimpan 1/3 · SA_seed-42
```


HC: Objective dari 3 Percobaan
Shipping fee (juta rupiah)
Iterasi (0 = state awal)
Percobaan 1 (seed 42)
Percobaan 2 (seed 43)
Percobaan 3 (seed 44)

SA

```
SA — RUN 1 — SEED 42

STATE AWAL
V001: tidak dimuat
V002: tidak dimuat
V003: tidak dimuat
V004: tidak dimuat
V005: (1, 0), orientasi=horizontal
V006: (6, 1), orientasi=vertical
V007: tidak dimuat
V008: tidak dimuat
V009: tidak dimuat
V010: tidak dimuat
V011: (3, 1), orientasi=horizontal
V012: tidak dimuat
V013: tidak dimuat
V014: tidak dimuat
V015: (6, 4), orientasi=horizontal
V016: (2, 6), orientasi=vertical
V017: tidak dimuat
V018: tidak dimuat
V019: (0, 7), orientasi=horizontal
V020: tidak dimuat
V021: tidak dimuat
V022: (4, 8), orientasi=horizontal
V023: tidak dimuat
V024: (0, 3), orientasi=vertical
Objective: 1750.0
Total berat: 79 / 100
Valid: True

STATE AKHIR
V001: (10, 4), orientasi=vertical
V002: tidak dimuat
V003: tidak dimuat
V004: tidak dimuat
V005: (0, 7), orientasi=horizontal
V006: tidak dimuat
V007: tidak dimuat
V008: tidak dimuat
V009: (0, 0), orientasi=horizontal
V010: (4, 0), orientasi=vertical
V011: (10, 2), orientasi=horizontal
V012: tidak dimuat
V013: tidak dimuat
V014: tidak dimuat
V015: tidak dimuat
V016: tidak dimuat
V017: tidak dimuat
V018: tidak dimuat
V019: tidak dimuat
V020: (8, 5), orientasi=horizontal
V021: (4, 5), orientasi=vertical
V022: (2, 5), orientasi=vertical
V023: (7, 2), orientasi=horizontal
V024: (4, 2), orientasi=vertical
Objective: 3080.0
Total berat: 100 / 100
Valid: True
Alasan berhenti: iteration_limit
```

```
STATE TERBAIK SA
V001: (10, 0), orientasi=vertical
V002: tidak dimuat
V003: tidak dimuat
V004: tidak dimuat
V005: (10, 6), orientasi=horizontal
V006: tidak dimuat
V007: tidak dimuat
V008: tidak dimuat
V009: (3, 0), orientasi=vertical
V010: (0, 0), orientasi=horizontal
V011: (2, 3), orientasi=horizontal
V012: tidak dimuat
V013: tidak dimuat
V014: tidak dimuat
V015: tidak dimuat
V016: tidak dimuat
V017: tidak dimuat
V018: tidak dimuat
V019: tidak dimuat
V020: (2, 5), orientasi=horizontal
V021: (4, 5), orientasi=vertical
V022: (8, 6), orientasi=vertical
V023: (4, 2), orientasi=vertical
V024: (7, 2), orientasi=vertical
Objective: 3080.0
Total berat: 100 / 100
Valid: True
Objective terbaik: 3080.0
Frekuensi local optimum: 50
Langkah menurun diterima: 45
Kandidat ditolak: 1945
Gagal mencari tetangga: 1
Parameter suhu: {'T0': 100, 'alpha': 0.999, 'T_min': 0.1, 'max_iter': 5000, 'seed': 42}
Objective hasil: 3080.0
Jumlah iterasi: 5000
Durasi: 22.5841 detik
Menjalankan 2/3 · SA_seed=43
Tersimpan 2/3 · SA_seed=43

SA — RUN 2 — SEED 43

STATE AWAL
V001: (3, 0), orientasi=horizontal
V002: tidak dimuat
V003: (1, 2), orientasi=vertical
V004: tidak dimuat
V005: tidak dimuat
V006: tidak dimuat
V007: tidak dimuat
V008: (9, 0), orientasi=horizontal
V009: tidak dimuat
V010: tidak dimuat
V011: (1, 8), orientasi=vertical
V012: tidak dimuat
V013: tidak dimuat
V014: tidak dimuat
V015: tidak dimuat
V016: tidak dimuat
V017: tidak dimuat
V018: tidak dimuat
V019: (2, 6), orientasi=vertical
V020: (9, 4), orientasi=horizontal
V021: tidak dimuat
V022: (7, 5), orientasi=vertical
V023: (6, 2), orientasi=horizontal
V024: (4, 7), orientasi=vertical
Objective: 2150.0
Total berat: 86 / 100
Valid: True
```

```
STATE AKHIR
V001: (0, 5), orientasi=vertical
V002: tidak dimuat
V003: tidak dimuat
V004: tidak dimuat
V005: (9, 8), orientasi=vertical
V006: tidak dimuat
V007: tidak dimuat
V008: tidak dimuat
V009: (2, 8), orientasi=vertical
V010: (4, 0), orientasi=vertical
V011: (2, 6), orientasi=horizontal
V012: tidak dimuat
V013: (5, 2), orientasi=vertical
V014: tidak dimuat
V015: tidak dimuat
V016: tidak dimuat
V017: tidak dimuat
V018: tidak dimuat
V019: (7, 6), orientasi=vertical
V020: (0, 1), orientasi=vertical
V021: tidak dimuat
V022: (0, 3), orientasi=horizontal
V023: (4, 5), orientasi=horizontal
V024: (9, 5), orientasi=horizontal
Objective: 3060.0
Total berat: 98 / 100
Valid: True
Alasan berhenti: iteration_limit

STATE TERBAIK SA
V001: (0, 3), orientasi=horizontal
V002: tidak dimuat
V003: tidak dimuat
V004: tidak dimuat
V005: (2, 5), orientasi=horizontal
V006: tidak dimuat
V007: tidak dimuat
V008: tidak dimuat
V009: (4, 0), orientasi=vertical
V010: (2, 8), orientasi=vertical
V011: (7, 6), orientasi=horizontal
V012: tidak dimuat
V013: (5, 2), orientasi=vertical
V014: tidak dimuat
V015: tidak dimuat
V016: tidak dimuat
V017: tidak dimuat
V018: tidak dimuat
V019: (10, 8), orientasi=horizontal
V020: (0, 0), orientasi=vertical
V021: tidak dimuat
V022: (0, 5), orientasi=vertical
V023: (4, 5), orientasi=horizontal
V024: (9, 5), orientasi=vertical
Objective: 3060.0
Total berat: 98 / 100
Valid: True
Objective terbaik: 3060.0
Frekuensi local optimum: 15
Langkah menurun diterima: 25
Kandidat ditolak: 1670
Gagal mencari tetangga: 1
Parameter suhu: {'T0': 100, 'alpha': 0.999, 'T_min': 0.1, 'max_iter': 5000, 'seed': 43}
Objective hasil: 3060.0
Jumlah iterasi: 5000
Durasi: 28.0321 detik
Menjalankan 3/3 · SA_seed-44
Tersimpan 3/3 · SA_seed-44
```

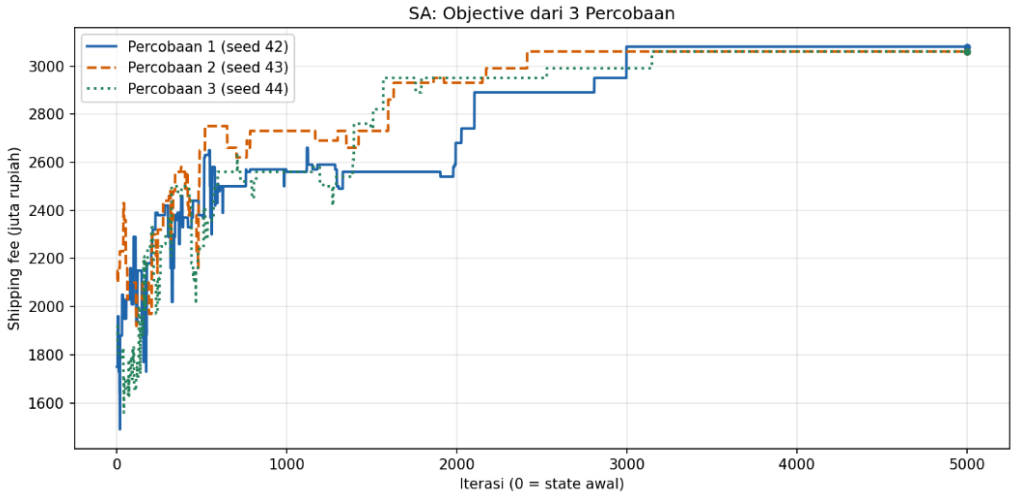
```
SA - RUN 3 - SEED 44

STATE AWAL
V001: tidak dimuat
V002: tidak dimuat
V003: tidak dimuat
V004: tidak dimuat
V005: (0, 0), orientasi=horizontal
V006: tidak dimuat
V007: (5, 5), orientasi=vertical
V008: tidak dimuat
V009: tidak dimuat
V010: tidak dimuat
V011: (1, 8), orientasi=horizontal
V012: (5, 1), orientasi=vertical
V013: tidak dimuat
V014: tidak dimuat
V015: (7, 7), orientasi=horizontal
V016: tidak dimuat
V017: tidak dimuat
V018: tidak dimuat
V019: (9, 1), orientasi=vertical
V020: (2, 0), orientasi=horizontal
V021: tidak dimuat
V022: (0, 6), orientasi=horizontal
V023: (4, 7), orientasi=horizontal
V024: tidak dimuat
Objective: 1920.0
Total berat: 78 / 100
Valid: True

STATE AKHIR
V001: (10, 5), orientasi=vertical
V002: tidak dimuat
V003: tidak dimuat
V004: tidak dimuat
V005: (2, 8), orientasi=vertical
V006: tidak dimuat
V007: tidak dimuat
V008: tidak dimuat
V009: (0, 3), orientasi=horizontal
V010: (4, 0), orientasi=vertical
V011: (3, 2), orientasi=horizontal
V012: tidak dimuat
V013: (5, 3), orientasi=horizontal
V014: tidak dimuat
V015: tidak dimuat
V016: tidak dimuat
V017: tidak dimuat
V018: tidak dimuat
V019: (8, 4), orientasi=vertical
V020: (8, 6), orientasi=horizontal
V021: tidak dimuat
V022: (8, 2), orientasi=horizontal
V023: (2, 4), orientasi=vertical
V024: (0, 0), orientasi=horizontal
Objective: 3060.0
Total berat: 98 / 100
Valid: True
Alasan berhenti: iteration_limit
```

```
STATE TERBAIK SA
V001: (10, 4), orientasi=vertical
V002: tidak dimuat
V003: tidak dimuat
V004: tidak dimuat
V005: (0, 0), orientasi=horizontal
V006: tidak dimuat
V007: tidak dimuat
V008: tidak dimuat
V009: (0, 3), orientasi=horizontal
V010: (4, 0), orientasi=vertical
V011: (8, 4), orientasi=vertical
V012: tidak dimuat
V013: (5, 3), orientasi=horizontal
V014: tidak dimuat
V015: tidak dimuat
V016: tidak dimuat
V017: tidak dimuat
V018: tidak dimuat
V019: (2, 1), orientasi=vertical
V020: (8, 6), orientasi=horizontal
V021: tidak dimuat
V022: (8, 2), orientasi=horizontal
V023: (2, 3), orientasi=vertical
V024: (2, 7), orientasi=vertical
Objective: 3060.0
Total berat: 98 / 100
Valid: True
Objective terbaik: 3060.0
Frekuensi local optimum: 31
Langkah menurun diterima: 43
Kandidat ditolak: 1727
Gagal mencari tetangga: 0
Parameter suhu: {'T0': 100, 'alpha': 0.999, 'T_min': 0.1, 'max_iter': 5000, 'seed': 44}
Objective hasil: 3060.0
Jumlah iterasi: 5000
Durasi: 30.8072 detik
Selesai. 3 percobaan sudah tersimpan.

SA: 3/3 run tersimpan.
Folder: /Users/acit/orca/workspaces/IF3070_G22_pararunten/daniel/output/simulated_annealing/sa_20261022_210053_124532
Sesi dibuat. Menyiapkan percobaan pertama.
Menjalankan 1/18 · A_pop-20_iter-50_seed-42
Tersimpan 1/18 · A_pop-20_iter-50_seed-42
```



GA

GA - RUN 1 - SEED 42

TERBAIK POPULASI AWAL
V001: (6, 3), orientasi=horizontal
V002: tidak dimuat

V003: tidak dimuat
V004: tidak dimuat
V005: (0, 0), orientasi=horizontal
V006: tidak dimuat
V007: tidak dimuat
V008: tidak dimuat
V009: tidak dimuat
V010: (3, 0), orientasi=horizontal
V011: (5, 1), orientasi=vertical
V012: tidak dimuat
V013: tidak dimuat
V014: tidak dimuat
V015: (1, 8), orientasi=horizontal
V016: tidak dimuat
V017: tidak dimuat
V018: tidak dimuat
V019: (0, 6), orientasi=horizontal
V020: (6, 6), orientasi=horizontal
V021: tidak dimuat
V022: (8, 5), orientasi=vertical
V023: (7, 0), orientasi=horizontal
V024: (0, 3), orientasi=vertical
Objective: 2460.0
Total berat: 88 / 100
Valid: True

TERBAIK SEPANJANG PENCARIAN
V001: tidak dimuat
V002: tidak dimuat
V003: tidak dimuat
V004: tidak dimuat
V005: (2, 0), orientasi=horizontal
V006: tidak dimuat
V007: tidak dimuat
V008: tidak dimuat
V009: (4, 1), orientasi=vertical
V010: (0, 2), orientasi=horizontal

V011: (7, 4), orientasi=horizontal
V012: tidak dimuat
V013: tidak dimuat
V014: tidak dimuat
V015: (2, 3), orientasi=horizontal
V016: tidak dimuat
V017: tidak dimuat
V018: tidak dimuat
V019: tidak dimuat
V020: (8, 6), orientasi=vertical
V021: (2, 6), orientasi=horizontal
V022: (7, 8), orientasi=horizontal
V023: (9, 3), orientasi=horizontal
V024: tidak dimuat
Objective: 2760.0
Total berat: 94 / 100
Valid: True
Kelompok: A
Jumlah populasi: 20
Peluang mutasi: 0.1
Maksimum populasi terakhir: 2340.0
Objective hasil: 2760.0
Jumlah iterasi: 50
Durasi: 3.9756 detik
Menjalankan 2/18 · A_pop-20_iter-50_seed-43
Tersimpan 2/18 · A_pop-20_iter-50_seed-43

GA - RUN 2 - SEED 43

TERBAIK POPULASI AWAL
V001: tidak dimuat
V002: tidak dimuat
V003: tidak dimuat
V004: tidak dimuat
V005: tidak dimuat
V006: tidak dimuat
V007: (3, 3), orientasi=vertical

V008: tidak dimuat
V009: (1, 8), orientasi=vertical
V010: tidak dimuat
V011: (10, 7), orientasi=horizontal
V012: tidak dimuat
V013: (4, 0), orientasi=vertical
V014: tidak dimuat
V015: tidak dimuat
V016: tidak dimuat
V017: tidak dimuat
V018: tidak dimuat
V019: (8, 7), orientasi=horizontal
V020: (8, 5), orientasi=vertical
V021: tidak dimuat
V022: (0, 0), orientasi=vertical
V023: (4, 5), orientasi=horizontal
V024: (0, 5), orientasi=vertical
Objective: 2390.0
Total berat: 79 / 100
Valid: True

TERBAIK SEPANJANG PENCARIAN
V001: tidak dimuat
V002: tidak dimuat
V003: tidak dimuat
V004: tidak dimuat
V005: (7, 8), orientasi=vertical
V006: tidak dimuat
V007: tidak dimuat
V008: (0, 6), orientasi=horizontal
V009: (0, 0), orientasi=vertical
V010: (10, 1), orientasi=horizontal
V011: (4, 3), orientasi=horizontal
V012: tidak dimuat
V013: tidak dimuat
V014: tidak dimuat
V015: tidak dimuat

V016: tidak dimuat
V017: tidak dimuat
V018: tidak dimuat
V019: tidak dimuat
V020: (0, 2), orientasi=vertical
V021: tidak dimuat
V022: (3, 5), orientasi=vertical
V023: (6, 3), orientasi=vertical
V024: (7, 0), orientasi=horizontal
Objective: 2690.0
Total berat: 99 / 100
Valid: True
Kelompok: A
Jumlah populasi: 20
Peluang mutasi: 0.1
Maksimum populasi terakhir: 2620.0
Objective hasil: 2690.0
Jumlah iterasi: 50
Durasi: 4.0259 detik
Menjalankan 3/18 · A_pop-20_iter-50_seed-44
Tersimpan 3/18 · A_pop-20_iter-50_seed-44

GA - RUN 3 - SEED 44

TERBAIK POPULASI AWAL
V001: tidak dimuat
V002: tidak dimuat
V003: tidak dimuat
V004: tidak dimuat
V005: (10, 7), orientasi=horizontal
V006: tidak dimuat
V007: tidak dimuat
V008: tidak dimuat
V009: tidak dimuat
V010: (0, 0), orientasi=horizontal
V011: (8, 6), orientasi=horizontal
V012: tidak dimuat

V013: tidak dimuat
V014: tidak dimuat
V015: (1, 8), orientasi=horizontal
V016: tidak dimuat
V017: (2, 1), orientasi=horizontal
V018: tidak dimuat
V019: tidak dimuat
V020: (6, 6), orientasi=horizontal
V021: (6, 0), orientasi=vertical
V022: (10, 2), orientasi=vertical
V023: tidak dimuat
V024: tidak dimuat
Objective: 2360.0
Total berat: 77 / 100
Valid: True

TERBAIK SEPANJANG PENCARIAN
V001: (8, 2), orientasi=vertical
V002: tidak dimuat
V003: tidak dimuat
V004: tidak dimuat
V005: (0, 0), orientasi=horizontal
V006: tidak dimuat
V007: (10, 2), orientasi=horizontal
V008: tidak dimuat
V009: (0, 3), orientasi=horizontal
V010: (6, 0), orientasi=horizontal
V011: (8, 8), orientasi=horizontal
V012: tidak dimuat
V013: tidak dimuat
V014: tidak dimuat
V015: (4, 4), orientasi=vertical
V016: tidak dimuat
V017: tidak dimuat
V018: tidak dimuat
V019: (10, 0), orientasi=vertical
V020: (4, 8), orientasi=vertical

V021: tidak dimuat
V022: (2, 6), orientasi=vertical
V023: tidak dimuat
V024: (2, 1), orientasi=horizontal
Objective: 2710.0
Total berat: 95 / 100
Valid: True
Kelompok: A
Jumlah populasi: 20
Peluang mutasi: 0.1
Maksimum populasi terakhir: 2360.0
Objective hasil: 2710.0
Jumlah iterasi: 50
Durasi: 4.0017 detik
Menjalankan 4/18 · A_pop-20_iter-100_seed-42
Tersimpan 4/18 · A_pop-20_iter-100_seed-42

GA - RUN 4 - SEED 42

TERBAIK POPULASI AWAL
V001: (6, 3), orientasi=horizontal
V002: tidak dimuat
V003: tidak dimuat
V004: tidak dimuat
V005: (0, 0), orientasi=horizontal
V006: tidak dimuat
V007: tidak dimuat
V008: tidak dimuat
V009: tidak dimuat
V010: (3, 0), orientasi=horizontal
V011: (5, 1), orientasi=vertical
V012: tidak dimuat
V013: tidak dimuat
V014: tidak dimuat
V015: (1, 8), orientasi=horizontal
V016: tidak dimuat
V017: tidak dimuat

V018: tidak dimuat
V019: (0, 6), orientasi=horizontal
V020: (6, 6), orientasi=horizontal
V021: tidak dimuat
V022: (8, 5), orientasi=vertical
V023: (7, 0), orientasi=horizontal
V024: (0, 3), orientasi=vertical
Objective: 2460.0
Total berat: 88 / 100
Valid: True

TERBAIK SEPANJANG PENCARIAN
V001: tidak dimuat
V002: tidak dimuat
V003: tidak dimuat
V004: tidak dimuat
V005: (2, 0), orientasi=horizontal
V006: tidak dimuat
V007: tidak dimuat
V008: tidak dimuat
V009: (4, 1), orientasi=vertical
V010: (0, 2), orientasi=horizontal
V011: (7, 4), orientasi=horizontal
V012: tidak dimuat
V013: tidak dimuat
V014: tidak dimuat
V015: (2, 3), orientasi=horizontal
V016: tidak dimuat
V017: tidak dimuat
V018: tidak dimuat
V019: tidak dimuat
V020: (8, 6), orientasi=vertical
V021: (2, 6), orientasi=horizontal
V022: (7, 8), orientasi=horizontal
V023: (9, 3), orientasi=horizontal
V024: tidak dimuat
Objective: 2760.0

Total berat: 94 / 100
Valid: True
Kelompok: A
Jumlah populasi: 20
Peluang mutasi: 0.1
Maksimum populasi terakhir: 2540.0
Objective hasil: 2760.0
Jumlah iterasi: 100
Durasi: 7.7995 detik
Menjalankan 5/18 · A_pop-20_iter-100_seed-43
Tersimpan 5/18 · A_pop-20_iter-100_seed-43

GA - RUN 5 - SEED 43

TERBAIK POPULASI AWAL
V001: tidak dimuat
V002: tidak dimuat
V003: tidak dimuat
V004: tidak dimuat
V005: tidak dimuat
V006: tidak dimuat
V007: (3, 3), orientasi=vertical
V008: tidak dimuat
V009: (1, 8), orientasi=vertical
V010: tidak dimuat
V011: (10, 7), orientasi=horizontal
V012: tidak dimuat
V013: (4, 0), orientasi=vertical
V014: tidak dimuat
V015: tidak dimuat
V016: tidak dimuat
V017: tidak dimuat
V018: tidak dimuat
V019: (8, 7), orientasi=horizontal
V020: (8, 5), orientasi=vertical
V021: tidak dimuat
V022: (0, 0), orientasi=vertical

V023: (4, 5), orientasi=horizontal
V024: (0, 5), orientasi=vertical
Objective: 2390.0
Total berat: 79 / 100
Valid: True

TERBAIK SEPANJANG PENCARIAN
V001: tidak dimuat
V002: tidak dimuat
V003: tidak dimuat
V004: tidak dimuat
V005: (7, 8), orientasi=vertical
V006: tidak dimuat
V007: tidak dimuat
V008: (0, 6), orientasi=horizontal
V009: (0, 0), orientasi=vertical
V010: (10, 1), orientasi=horizontal
V011: (4, 3), orientasi=horizontal
V012: tidak dimuat
V013: tidak dimuat
V014: tidak dimuat
V015: tidak dimuat
V016: tidak dimuat
V017: tidak dimuat
V018: tidak dimuat
V019: tidak dimuat
V020: (0, 2), orientasi=vertical
V021: tidak dimuat
V022: (3, 5), orientasi=vertical
V023: (6, 3), orientasi=vertical
V024: (7, 0), orientasi=horizontal
Objective: 2690.0
Total berat: 99 / 100
Valid: True
Kelompok: A
Jumlah populasi: 20
Peluang mutasi: 0.1

Maksimum populasi terakhir: 2500.0
Objective hasil: 2690.0
Jumlah iterasi: 100
Durasi: 7.7844 detik
Menjalankan 6/18 · A_pop-20_iter-100_seed-44
Tersimpan 6/18 · A_pop-20_iter-100_seed-44

GA - RUN 6 - SEED 44

TERBAIK POPULASI AWAL
V001: tidak dimuat
V002: tidak dimuat
V003: tidak dimuat
V004: tidak dimuat
V005: (10, 7), orientasi=horizontal
V006: tidak dimuat
V007: tidak dimuat
V008: tidak dimuat
V009: tidak dimuat
V010: (0, 0), orientasi=horizontal
V011: (8, 6), orientasi=horizontal
V012: tidak dimuat
V013: tidak dimuat
V014: tidak dimuat
V015: (1, 8), orientasi=horizontal
V016: tidak dimuat
V017: (2, 1), orientasi=horizontal
V018: tidak dimuat
V019: tidak dimuat
V020: (6, 6), orientasi=horizontal
V021: (6, 0), orientasi=vertical
V022: (10, 2), orientasi=vertical
V023: tidak dimuat
V024: tidak dimuat
Objective: 2360.0
Total berat: 77 / 100
Valid: True

TERBAIK SEPANJANG PENCARIAN
V001: (0, 0), orientasi=vertical
V002: tidak dimuat
V003: tidak dimuat
V004: tidak dimuat
V005: (0, 7), orientasi=horizontal
V006: tidak dimuat
V007: tidak dimuat
V008: tidak dimuat
V009: (3, 4), orientasi=vertical
V010: (2, 2), orientasi=vertical
V011: (5, 6), orientasi=vertical
V012: tidak dimuat
V013: tidak dimuat
V014: tidak dimuat
V015: (10, 6), orientasi=vertical
V016: (3, 0), orientasi=vertical
V017: tidak dimuat
V018: tidak dimuat
V019: tidak dimuat
V020: (10, 2), orientasi=horizontal
V021: tidak dimuat
V022: (2, 6), orientasi=vertical
V023: (7, 7), orientasi=horizontal
V024: tidak dimuat
Objective: 2730.0
Total berat: 96 / 100
Valid: True
Kelompok: A
Jumlah populasi: 20
Peluang mutasi: 0.1
Maksimum populasi terakhir: 2460.0
Objective hasil: 2730.0
Jumlah iterasi: 100
Durasi: 7.8928 detik
Menjalankan 7/18 · A_pop-20_iter-200_seed-42

Tersimpan 7/18 · A_pop-20_iter-200_seed-42

GA - RUN 7 - SEED 42

TERBAIK POPULASI AWAL
V001: (6, 3), orientasi=horizontal
V002: tidak dimuat
V003: tidak dimuat
V004: tidak dimuat
V005: (0, 0), orientasi=horizontal
V006: tidak dimuat
V007: tidak dimuat
V008: tidak dimuat
V009: tidak dimuat
V010: (3, 0), orientasi=horizontal
V011: (5, 1), orientasi=vertical
V012: tidak dimuat
V013: tidak dimuat
V014: tidak dimuat
V015: (1, 8), orientasi=horizontal
V016: tidak dimuat
V017: tidak dimuat
V018: tidak dimuat
V019: (0, 6), orientasi=horizontal
V020: (6, 6), orientasi=horizontal
V021: tidak dimuat
V022: (8, 5), orientasi=vertical
V023: (7, 0), orientasi=horizontal
V024: (0, 3), orientasi=vertical
Objective: 2460.0
Total berat: 88 / 100
Valid: True

TERBAIK SEPANJANG PENCARIAN
V001: tidak dimuat
V002: tidak dimuat
V003: tidak dimuat

V004: tidak dimuat
V005: (2, 0), orientasi=horizontal
V006: tidak dimuat
V007: tidak dimuat
V008: tidak dimuat
V009: (4, 1), orientasi=vertical
V010: (0, 2), orientasi=horizontal
V011: (7, 4), orientasi=horizontal
V012: tidak dimuat
V013: tidak dimuat
V014: tidak dimuat
V015: (2, 3), orientasi=horizontal
V016: tidak dimuat
V017: tidak dimuat
V018: tidak dimuat
V019: tidak dimuat
V020: (8, 6), orientasi=vertical
V021: (2, 6), orientasi=horizontal
V022: (7, 8), orientasi=horizontal
V023: (9, 3), orientasi=horizontal
V024: tidak dimuat
Objective: 2760.0
Total berat: 94 / 100
Valid: True
Kelompok: A
Jumlah populasi: 20
Peluang mutasi: 0.1
Maksimum populasi terakhir: 2450.0
Objective hasil: 2760.0
Jumlah iterasi: 200
Durasi: 15.4784 detik
Menjalankan 8/18 · A_pop-20_iter-200_seed-43
Tersimpan 8/18 · A_pop-20_iter-200_seed-43

GA - RUN 8 - SEED 43

TERBAIK POPULASI AWAL

V001: tidak dimuat
V002: tidak dimuat
V003: tidak dimuat
V004: tidak dimuat
V005: tidak dimuat
V006: tidak dimuat
V007: (3, 3), orientasi=vertical
V008: tidak dimuat
V009: (1, 8), orientasi=vertical
V010: tidak dimuat
V011: (10, 7), orientasi=horizontal
V012: tidak dimuat
V013: (4, 0), orientasi=vertical
V014: tidak dimuat
V015: tidak dimuat
V016: tidak dimuat
V017: tidak dimuat
V018: tidak dimuat
V019: (8, 7), orientasi=horizontal
V020: (8, 5), orientasi=vertical
V021: tidak dimuat
V022: (0, 0), orientasi=vertical
V023: (4, 5), orientasi=horizontal
V024: (0, 5), orientasi=vertical
Objective: 2390.0
Total berat: 79 / 100
Valid: True

TERBAIK SEPANJANG PENCARIAN
V001: (7, 8), orientasi=horizontal
V002: tidak dimuat
V003: (0, 0), orientasi=horizontal
V004: tidak dimuat
V005: (7, 0), orientasi=vertical
V006: tidak dimuat
V007: tidak dimuat
V008: tidak dimuat

V009: tidak dimuat
V010: tidak dimuat
V011: (6, 3), orientasi=horizontal
V012: tidak dimuat
V013: tidak dimuat
V014: tidak dimuat
V015: (8, 4), orientasi=horizontal
V016: tidak dimuat
V017: tidak dimuat
V018: tidak dimuat
V019: tidak dimuat
V020: (8, 6), orientasi=vertical
V021: (3, 5), orientasi=vertical
V022: (8, 2), orientasi=horizontal
V023: (0, 6), orientasi=vertical
V024: (4, 0), orientasi=horizontal
Objective: 2710.0
Total berat: 99 / 100
Valid: True
Kelompok: A
Jumlah populasi: 20
Peluang mutasi: 0.1
Maksimum populasi terakhir: 2340.0
Objective hasil: 2710.0
Jumlah iterasi: 200
Durasi: 15.6266 detik
Menjalankan 9/18 · A_pop-20_iter-200_seed-44
Tersimpan 9/18 · A_pop-20_iter-200_seed-44

GA - RUN 9 - SEED 44

TERBAIK POPULASI AWAL
V001: tidak dimuat
V002: tidak dimuat
V003: tidak dimuat
V004: tidak dimuat
V005: (10, 7), orientasi=horizontal

V006: tidak dimuat
V007: tidak dimuat
V008: tidak dimuat
V009: tidak dimuat
V010: (0, 0), orientasi=horizontal
V011: (8, 6), orientasi=horizontal
V012: tidak dimuat
V013: tidak dimuat
V014: tidak dimuat
V015: (1, 8), orientasi=horizontal
V016: tidak dimuat
V017: (2, 1), orientasi=horizontal
V018: tidak dimuat
V019: tidak dimuat
V020: (6, 6), orientasi=horizontal
V021: (6, 0), orientasi=vertical
V022: (10, 2), orientasi=vertical
V023: tidak dimuat
V024: tidak dimuat
Objective: 2360.0
Total berat: 77 / 100
Valid: True

TERBAIK SEPANJANG PENCARIAN
V001: (0, 0), orientasi=vertical
V002: tidak dimuat
V003: tidak dimuat
V004: tidak dimuat
V005: (0, 7), orientasi=horizontal
V006: tidak dimuat
V007: tidak dimuat
V008: tidak dimuat
V009: (3, 4), orientasi=vertical
V010: (2, 2), orientasi=vertical
V011: (5, 6), orientasi=vertical
V012: tidak dimuat
V013: tidak dimuat

V014: tidak dimuat
V015: (10, 6), orientasi=vertical
V016: (3, 0), orientasi=vertical
V017: tidak dimuat
V018: tidak dimuat
V019: tidak dimuat
V020: (10, 2), orientasi=horizontal
V021: tidak dimuat
V022: (2, 6), orientasi=vertical
V023: (7, 7), orientasi=horizontal
V024: tidak dimuat
Objective: 2730.0
Total berat: 96 / 100
Valid: True
Kelompok: A
Jumlah populasi: 20
Peluang mutasi: 0.1
Maksimum populasi terakhir: 2700.0
Objective hasil: 2730.0
Jumlah iterasi: 200
Durasi: 15.5839 detik
Menjalankan 10/18 · B_pop-10_iter-100_seed-42
Tersimpan 10/18 · B_pop-10_iter-100_seed-42

GA - RUN 10 - SEED 42

TERBAIK POPULASI AWAL
V001: (6, 3), orientasi=horizontal
V002: tidak dimuat
V003: tidak dimuat
V004: tidak dimuat
V005: (0, 0), orientasi=horizontal
V006: tidak dimuat
V007: tidak dimuat
V008: tidak dimuat
V009: tidak dimuat
V010: (3, 0), orientasi=horizontal

V011: (5, 1), orientasi=vertical
V012: tidak dimuat
V013: tidak dimuat
V014: tidak dimuat
V015: (1, 8), orientasi=horizontal
V016: tidak dimuat
V017: tidak dimuat
V018: tidak dimuat
V019: (0, 6), orientasi=horizontal
V020: (6, 6), orientasi=horizontal
V021: tidak dimuat
V022: (8, 5), orientasi=vertical
V023: (7, 0), orientasi=horizontal
V024: (0, 3), orientasi=vertical
Objective: 2460.0
Total berat: 88 / 100
Valid: True

TERBAIK SEPANJANG PENCARIAN
V001: tidak dimuat
V002: tidak dimuat
V003: (6, 3), orientasi=horizontal
V004: tidak dimuat
V005: (5, 0), orientasi=horizontal
V006: tidak dimuat
V007: tidak dimuat
V008: tidak dimuat
V009: (10, 3), orientasi=horizontal
V010: tidak dimuat
V011: (8, 8), orientasi=horizontal
V012: tidak dimuat
V013: tidak dimuat
V014: tidak dimuat
V015: tidak dimuat
V016: tidak dimuat
V017: tidak dimuat
V018: tidak dimuat

V019: (0, 8), orientasi=vertical
V020: (2, 8), orientasi=vertical
V021: (1, 0), orientasi=vertical
V022: (1, 6), orientasi=horizontal
V023: (7, 0), orientasi=horizontal
V024: tidak dimuat
Objective: 2670.0
Total berat: 95 / 100
Valid: True
Kelompok: B
Jumlah populasi: 10
Peluang mutasi: 0.1
Maksimum populasi terakhir: 2240.0
Objective hasil: 2670.0
Jumlah iterasi: 100
Durasi: 3.7059 detik
Menjalankan 11/18 · B_pop-10_iter-100_seed-43
Tersimpan 11/18 · B_pop-10_iter-100_seed-43

GA - RUN 11 - SEED 43

TERBAIK POPULASI AWAL
V001: tidak dimuat
V002: tidak dimuat
V003: tidak dimuat
V004: tidak dimuat
V005: (10, 4), orientasi=horizontal
V006: tidak dimuat
V007: (5, 0), orientasi=horizontal
V008: (0, 6), orientasi=horizontal
V009: tidak dimuat
V010: tidak dimuat
V011: (10, 7), orientasi=horizontal
V012: tidak dimuat
V013: tidak dimuat
V014: tidak dimuat
V015: tidak dimuat

V016: tidak dimuat
V017: tidak dimuat
V018: tidak dimuat
V019: (3, 1), orientasi=vertical
V020: (7, 4), orientasi=horizontal
V021: (7, 0), orientasi=horizontal
V022: tidak dimuat
V023: (0, 0), orientasi=horizontal
V024: (3, 7), orientasi=vertical
Objective: 2160.0
Total berat: 91 / 100
Valid: True

TERBAIK SEPANJANG PENCARIAN
V001: tidak dimuat
V002: tidak dimuat
V003: tidak dimuat
V004: tidak dimuat
V005: (1, 8), orientasi=vertical
V006: tidak dimuat
V007: tidak dimuat
V008: tidak dimuat
V009: tidak dimuat
V010: (10, 0), orientasi=horizontal
V011: (1, 2), orientasi=vertical
V012: tidak dimuat
V013: tidak dimuat
V014: tidak dimuat
V015: (0, 4), orientasi=vertical
V016: (8, 2), orientasi=horizontal
V017: tidak dimuat
V018: tidak dimuat
V019: (6, 0), orientasi=vertical
V020: (2, 4), orientasi=horizontal
V021: (4, 2), orientasi=vertical
V022: (0, 0), orientasi=horizontal
V023: (4, 7), orientasi=horizontal

V024: tidak dimuat
Objective: 2670.0
Total berat: 90 / 100
Valid: True
Kelompok: B
Jumlah populasi: 10
Peluang mutasi: 0.1
Maksimum populasi terakhir: 2300.0
Objective hasil: 2670.0
Jumlah iterasi: 100
Durasi: 3.5372 detik
Menjalankan 12/18 · B_pop-10_iter-100_seed-44
Tersimpan 12/18 · B_pop-10_iter-100_seed-44

GA - RUN 12 - SEED 44

TERBAIK POPULASI AWAL
V001: tidak dimuat
V002: tidak dimuat
V003: (5, 0), orientasi=horizontal
V004: tidak dimuat
V005: (3, 6), orientasi=horizontal
V006: tidak dimuat
V007: (5, 6), orientasi=vertical
V008: tidak dimuat
V009: tidak dimuat
V010: tidak dimuat
V011: (8, 8), orientasi=vertical
V012: tidak dimuat
V013: tidak dimuat
V014: tidak dimuat
V015: (0, 0), orientasi=horizontal
V016: tidak dimuat
V017: tidak dimuat
V018: tidak dimuat
V019: (10, 0), orientasi=vertical
V020: tidak dimuat

V021: (0, 2), orientasi=horizontal
V022: (10, 2), orientasi=vertical
V023: tidak dimuat
V024: (0, 6), orientasi=vertical
Objective: 2310.0
Total berat: 86 / 100
Valid: True

TERBAIK SEPANJANG PENCARIAN
V001: (2, 5), orientasi=vertical
V002: tidak dimuat
V003: tidak dimuat
V004: tidak dimuat
V005: (0, 1), orientasi=horizontal
V006: tidak dimuat
V007: tidak dimuat
V008: tidak dimuat
V009: (8, 3), orientasi=horizontal
V010: (10, 0), orientasi=horizontal
V011: (4, 0), orientasi=vertical
V012: tidak dimuat
V013: (4, 3), orientasi=horizontal
V014: tidak dimuat
V015: (2, 1), orientasi=vertical
V016: tidak dimuat
V017: tidak dimuat
V018: tidak dimuat
V019: (0, 4), orientasi=horizontal
V020: tidak dimuat
V021: tidak dimuat
V022: (0, 6), orientasi=vertical
V023: (6, 0), orientasi=horizontal
V024: tidak dimuat
Objective: 2670.0
Total berat: 94 / 100
Valid: True
Kelompok: B

Jumlah populasi: 10
Peluang mutasi: 0.1
Maksimum populasi terakhir: 2220.0
Objective hasil: 2670.0
Jumlah iterasi: 100
Durasi: 3.8657 detik
Menjalankan 13/18 · B_pop-20_iter-100_seed-42
Tersimpan 13/18 · B_pop-20_iter-100_seed-42

GA - RUN 13 - SEED 42

TERBAIK POPULASI AWAL
V001: (6, 3), orientasi=horizontal
V002: tidak dimuat
V003: tidak dimuat
V004: tidak dimuat
V005: (0, 0), orientasi=horizontal
V006: tidak dimuat
V007: tidak dimuat
V008: tidak dimuat
V009: tidak dimuat
V010: (3, 0), orientasi=horizontal
V011: (5, 1), orientasi=vertical
V012: tidak dimuat
V013: tidak dimuat
V014: tidak dimuat
V015: (1, 8), orientasi=horizontal
V016: tidak dimuat
V017: tidak dimuat
V018: tidak dimuat
V019: (0, 6), orientasi=horizontal
V020: (6, 6), orientasi=horizontal
V021: tidak dimuat
V022: (8, 5), orientasi=vertical
V023: (7, 0), orientasi=horizontal
V024: (0, 3), orientasi=vertical
Objective: 2460.0

Total berat: 88 / 100
Valid: True

TERBAIK SEPANJANG PENCARIAN
V001: tidak dimuat
V002: tidak dimuat
V003: tidak dimuat
V004: tidak dimuat
V005: (2, 0), orientasi=horizontal
V006: tidak dimuat
V007: tidak dimuat
V008: tidak dimuat
V009: (4, 1), orientasi=vertical
V010: (0, 2), orientasi=horizontal
V011: (7, 4), orientasi=horizontal
V012: tidak dimuat
V013: tidak dimuat
V014: tidak dimuat
V015: (2, 3), orientasi=horizontal
V016: tidak dimuat
V017: tidak dimuat
V018: tidak dimuat
V019: tidak dimuat
V020: (8, 6), orientasi=vertical
V021: (2, 6), orientasi=horizontal
V022: (7, 8), orientasi=horizontal
V023: (9, 3), orientasi=horizontal
V024: tidak dimuat
Objective: 2760.0
Total berat: 94 / 100
Valid: True
Kelompok: B
Jumlah populasi: 20
Peluang mutasi: 0.1
Maksimum populasi terakhir: 2540.0
Objective hasil: 2760.0
Jumlah iterasi: 100

Durasi: 7.8116 detik
Menjalankan 14/18 · B_pop-20_iter-100_seed-43
Tersimpan 14/18 · B_pop-20_iter-100_seed-43

GA - RUN 14 - SEED 43

TERBAIK POPULASI AWAL
V001: tidak dimuat
V002: tidak dimuat
V003: tidak dimuat
V004: tidak dimuat
V005: tidak dimuat
V006: tidak dimuat
V007: (3, 3), orientasi=vertical
V008: tidak dimuat
V009: (1, 8), orientasi=vertical
V010: tidak dimuat
V011: (10, 7), orientasi=horizontal
V012: tidak dimuat
V013: (4, 0), orientasi=vertical
V014: tidak dimuat
V015: tidak dimuat
V016: tidak dimuat
V017: tidak dimuat
V018: tidak dimuat
V019: (8, 7), orientasi=horizontal
V020: (8, 5), orientasi=vertical
V021: tidak dimuat
V022: (0, 0), orientasi=vertical
V023: (4, 5), orientasi=horizontal
V024: (0, 5), orientasi=vertical
Objective: 2390.0
Total berat: 79 / 100
Valid: True

TERBAIK SEPANJANG PENCARIAN
V001: tidak dimuat

V002: tidak dimuat
V003: tidak dimuat
V004: tidak dimuat
V005: (7, 8), orientasi=vertical
V006: tidak dimuat
V007: tidak dimuat
V008: (0, 6), orientasi=horizontal
V009: (0, 0), orientasi=vertical
V010: (10, 1), orientasi=horizontal
V011: (4, 3), orientasi=horizontal
V012: tidak dimuat
V013: tidak dimuat
V014: tidak dimuat
V015: tidak dimuat
V016: tidak dimuat
V017: tidak dimuat
V018: tidak dimuat
V019: tidak dimuat
V020: (0, 2), orientasi=vertical
V021: tidak dimuat
V022: (3, 5), orientasi=vertical
V023: (6, 3), orientasi=vertical
V024: (7, 0), orientasi=horizontal
Objective: 2690.0
Total berat: 99 / 100
Valid: True
Kelompok: B
Jumlah populasi: 20
Peluang mutasi: 0.1
Maksimum populasi terakhir: 2500.0
Objective hasil: 2690.0
Jumlah iterasi: 100
Durasi: 7.7917 detik
Menjalankan 15/18 · B_pop-20_iter-100_seed-44
Tersimpan 15/18 · B_pop-20_iter-100_seed-44

GA - RUN 15 - SEED 44

TERBAIK POPULASI AWAL
V001: tidak dimuat
V002: tidak dimuat
V003: tidak dimuat
V004: tidak dimuat
V005: (10, 7), orientasi=horizontal
V006: tidak dimuat
V007: tidak dimuat
V008: tidak dimuat
V009: tidak dimuat
V010: (0, 0), orientasi=horizontal
V011: (8, 6), orientasi=horizontal
V012: tidak dimuat
V013: tidak dimuat
V014: tidak dimuat
V015: (1, 8), orientasi=horizontal
V016: tidak dimuat
V017: (2, 1), orientasi=horizontal
V018: tidak dimuat
V019: tidak dimuat
V020: (6, 6), orientasi=horizontal
V021: (6, 0), orientasi=vertical
V022: (10, 2), orientasi=vertical
V023: tidak dimuat
V024: tidak dimuat
Objective: 2360.0
Total berat: 77 / 100
Valid: True

TERBAIK SEPANJANG PENCARIAN
V001: (0, 0), orientasi=vertical
V002: tidak dimuat
V003: tidak dimuat
V004: tidak dimuat
V005: (0, 7), orientasi=horizontal
V006: tidak dimuat

V007: tidak dimuat
V008: tidak dimuat
V009: (3, 4), orientasi=vertical
V010: (2, 2), orientasi=vertical
V011: (5, 6), orientasi=vertical
V012: tidak dimuat
V013: tidak dimuat
V014: tidak dimuat
V015: (10, 6), orientasi=vertical
V016: (3, 0), orientasi=vertical
V017: tidak dimuat
V018: tidak dimuat
V019: tidak dimuat
V020: (10, 2), orientasi=horizontal
V021: tidak dimuat
V022: (2, 6), orientasi=vertical
V023: (7, 7), orientasi=horizontal
V024: tidak dimuat
Objective: 2730.0
Total berat: 96 / 100
Valid: True
Kelompok: B
Jumlah populasi: 20
Peluang mutasi: 0.1
Maksimum populasi terakhir: 2460.0
Objective hasil: 2730.0
Jumlah iterasi: 100
Durasi: 7.8433 detik
Menjalankan 16/18 · B_pop-40_iter-100_seed-42
Tersimpan 16/18 · B_pop-40_iter-100_seed-42

GA - RUN 16 - SEED 42

TERBAIK POPULASI AWAL
V001: (6, 3), orientasi=horizontal
V002: tidak dimuat
V003: tidak dimuat

V004: tidak dimuat
V005: (0, 0), orientasi=horizontal
V006: tidak dimuat
V007: tidak dimuat
V008: tidak dimuat
V009: tidak dimuat
V010: (3, 0), orientasi=horizontal
V011: (5, 1), orientasi=vertical
V012: tidak dimuat
V013: tidak dimuat
V014: tidak dimuat
V015: (1, 8), orientasi=horizontal
V016: tidak dimuat
V017: tidak dimuat
V018: tidak dimuat
V019: (0, 6), orientasi=horizontal
V020: (6, 6), orientasi=horizontal
V021: tidak dimuat
V022: (8, 5), orientasi=vertical
V023: (7, 0), orientasi=horizontal
V024: (0, 3), orientasi=vertical
Objective: 2460.0
Total berat: 88 / 100
Valid: True

TERBAIK SEPANJANG PENCARIAN
V001: (5, 8), orientasi=horizontal
V002: tidak dimuat
V003: tidak dimuat
V004: tidak dimuat
V005: (2, 7), orientasi=horizontal
V006: tidak dimuat
V007: tidak dimuat
V008: (7, 4), orientasi=horizontal
V009: (0, 0), orientasi=vertical
V010: (10, 1), orientasi=horizontal
V011: (4, 2), orientasi=vertical

V012: tidak dimuat
V013: tidak dimuat
V014: tidak dimuat
V015: tidak dimuat
V016: (0, 2), orientasi=horizontal
V017: tidak dimuat
V018: tidak dimuat
V019: tidak dimuat
V020: (2, 2), orientasi=horizontal
V021: tidak dimuat
V022: (7, 0), orientasi=vertical
V023: tidak dimuat
V024: (4, 4), orientasi=vertical
Objective: 2770.0
Total berat: 96 / 100
Valid: True
Kelompok: B
Jumlah populasi: 40
Peluang mutasi: 0.1
Maksimum populasi terakhir: 2630.0
Objective hasil: 2770.0
Jumlah iterasi: 100
Durasi: 15.9328 detik
Menjalankan 17/18 · B_pop-40_iter-100_seed-43
Tersimpan 17/18 · B_pop-40_iter-100_seed-43

GA - RUN 17 - SEED 43

TERBAIK POPULASI AWAL
V001: tidak dimuat
V002: tidak dimuat
V003: (0, 0), orientasi=vertical
V004: tidak dimuat
V005: (5, 2), orientasi=vertical
V006: tidak dimuat
V007: tidak dimuat
V008: tidak dimuat

V009: tidak dimuat
V010: (0, 6), orientasi=vertical
V011: (5, 8), orientasi=horizontal
V012: (8, 1), orientasi=vertical
V013: tidak dimuat
V014: tidak dimuat
V015: (8, 8), orientasi=horizontal
V016: tidak dimuat
V017: tidak dimuat
V018: tidak dimuat
V019: (5, 4), orientasi=horizontal
V020: (1, 8), orientasi=vertical
V021: tidak dimuat
V022: (0, 4), orientasi=horizontal
V023: tidak dimuat
V024: (8, 5), orientasi=vertical
Objective: 2510.0
Total berat: 89 / 100
Valid: True

TERBAIK SEPANJANG PENCARIAN
V001: tidak dimuat
V002: tidak dimuat
V003: tidak dimuat
V004: tidak dimuat
V005: (10, 0), orientasi=horizontal
V006: tidak dimuat
V007: tidak dimuat
V008: tidak dimuat
V009: (0, 0), orientasi=vertical
V010: tidak dimuat
V011: (0, 5), orientasi=vertical
V012: tidak dimuat
V013: (0, 2), orientasi=vertical
V014: tidak dimuat
V015: tidak dimuat
V016: tidak dimuat

V017: tidak dimuat
V018: tidak dimuat
V019: (6, 6), orientasi=vertical
V020: (5, 8), orientasi=vertical
V021: (8, 3), orientasi=vertical
V022: (3, 6), orientasi=vertical
V023: (0, 7), orientasi=vertical
V024: (7, 0), orientasi=horizontal
Objective: 2920.0
Total berat: 97 / 100
Valid: True
Kelompok: B
Jumlah populasi: 40
Peluang mutasi: 0.1
Maksimum populasi terakhir: 2400.0
Objective hasil: 2920.0
Jumlah iterasi: 100
Durasi: 16.3333 detik
Menjalankan 18/18 · B_pop-40_iter-100_seed-44
Tersimpan 18/18 · B_pop-40_iter-100_seed-44

GA - RUN 18 - SEED 44

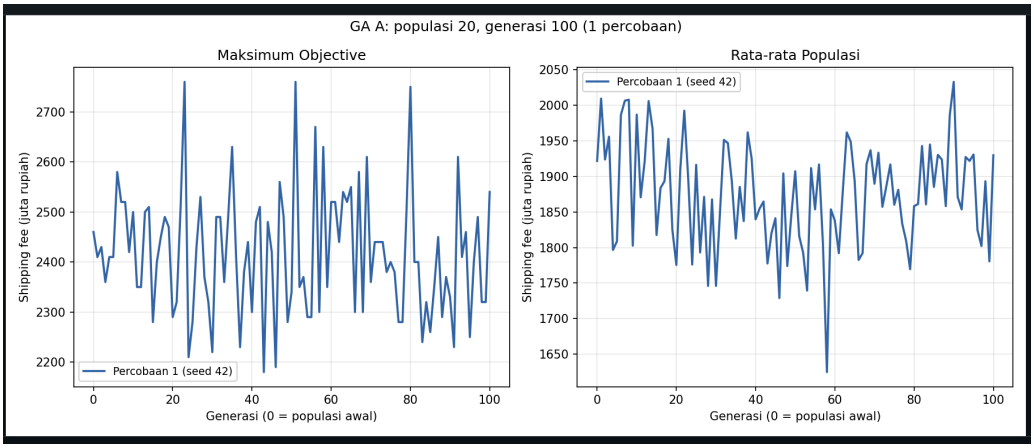
TERBAIK POPULASI AWAL
V001: tidak dimuat
V002: tidak dimuat
V003: tidak dimuat
V004: tidak dimuat
V005: (10, 7), orientasi=horizontal
V006: tidak dimuat
V007: tidak dimuat
V008: tidak dimuat
V009: tidak dimuat
V010: (0, 0), orientasi=horizontal
V011: (8, 6), orientasi=horizontal
V012: tidak dimuat
V013: tidak dimuat

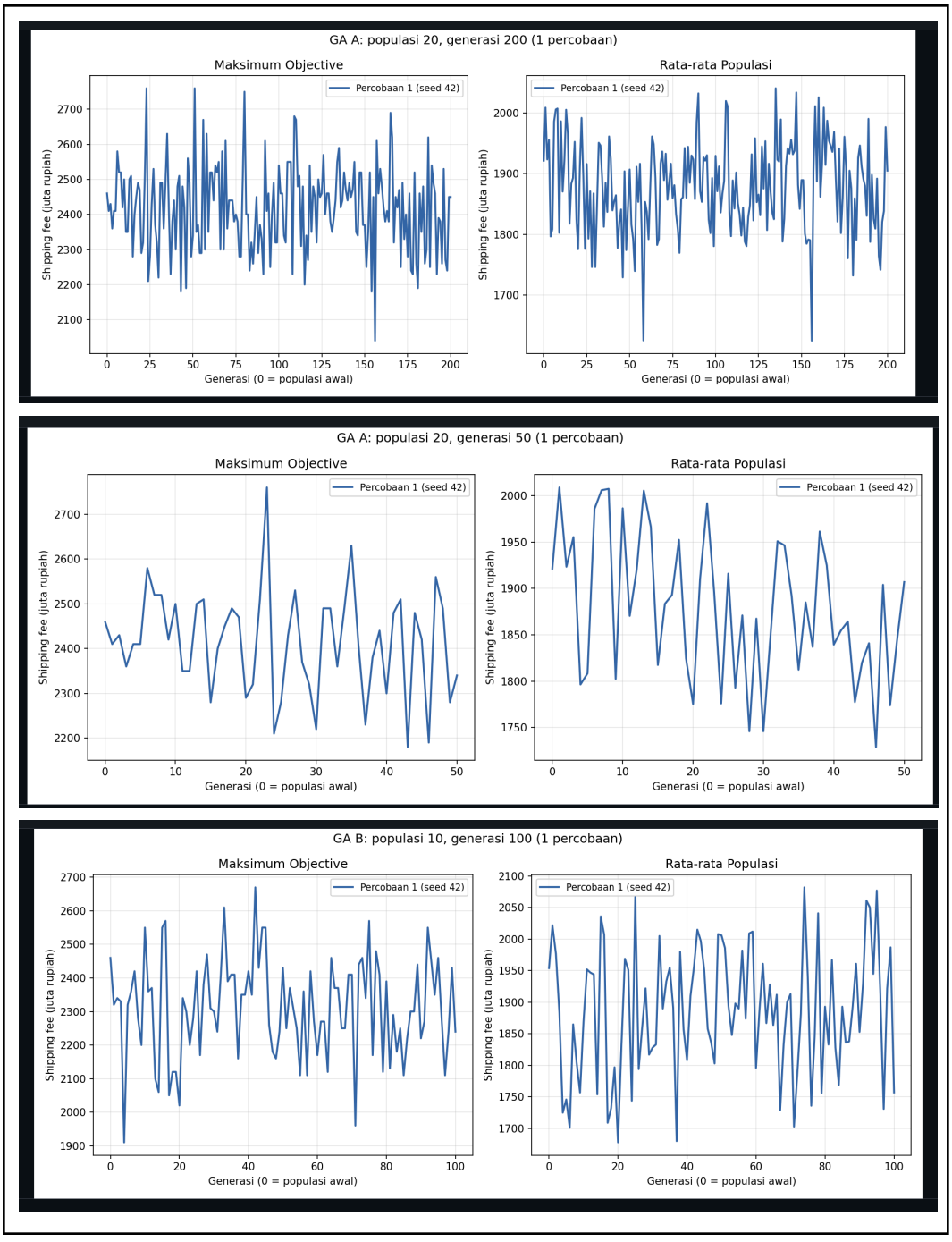
V014: tidak dimuat
V015: (1, 8), orientasi=horizontal
V016: tidak dimuat
V017: (2, 1), orientasi=horizontal
V018: tidak dimuat
V019: tidak dimuat
V020: (6, 6), orientasi=horizontal
V021: (6, 0), orientasi=vertical
V022: (10, 2), orientasi=vertical
V023: tidak dimuat
V024: tidak dimuat
Objective: 2360.0
Total berat: 77 / 100
Valid: True

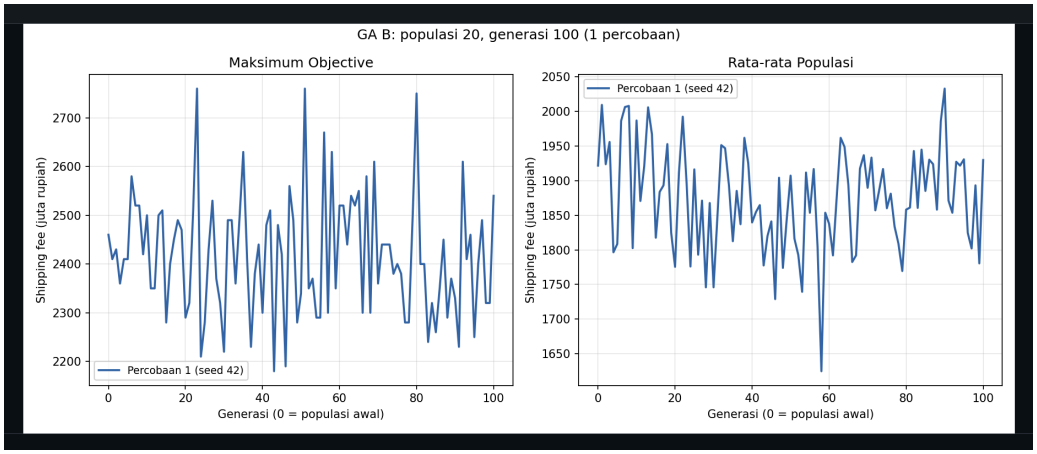
TERBAIK SEPANJANG PENCARIAN
V001: tidak dimuat
V002: tidak dimuat
V003: tidak dimuat
V004: tidak dimuat
V005: (1, 0), orientasi=vertical
V006: tidak dimuat
V007: (2, 4), orientasi=vertical
V008: tidak dimuat
V009: (0, 6), orientasi=vertical
V010: (4, 8), orientasi=vertical
V011: (7, 6), orientasi=horizontal
V012: tidak dimuat
V013: tidak dimuat
V014: tidak dimuat
V015: (0, 2), orientasi=vertical
V016: tidak dimuat
V017: tidak dimuat
V018: tidak dimuat
V019: tidak dimuat
V020: (0, 8), orientasi=vertical
V021: tidak dimuat

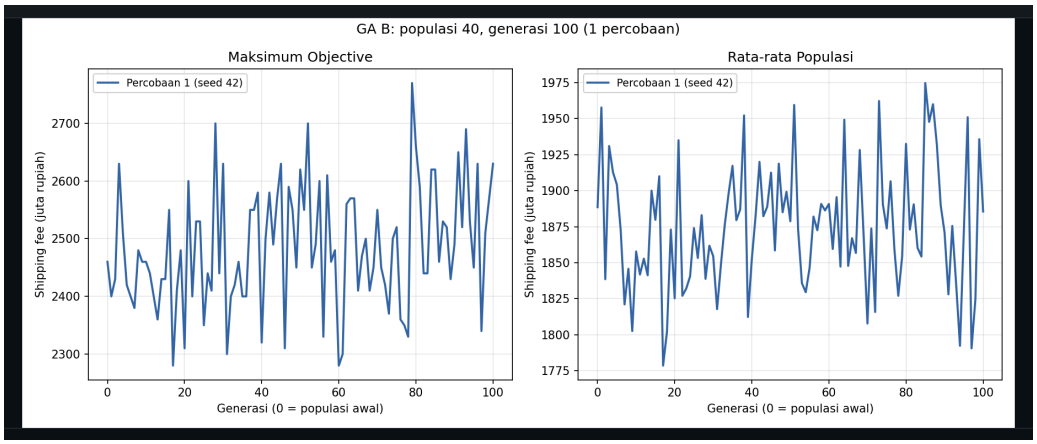
V022: (2, 2), orientasi=horizontal
V023: (9, 4), orientasi=vertical
V024: (6, 0), orientasi=horizontal
Objective: 2790.0
Total berat: 100 / 100
Valid: True
Kelompok: B
Jumlah populasi: 40
Peluang mutasi: 0.1
Maksimum populasi terakhir: 2530.0
Objective hasil: 2790.0
Jumlah iterasi: 100
Durasi: 16.4617 detik
Selesai. 18 percobaan sudah tersimpan.

GA: 18/18 run tersimpan.
Folder:
/Users/acit/orca/workspaces/IF3070_G22_pararunten/daniel/output/genetic_algorithm/ga_20261022_210215_376173



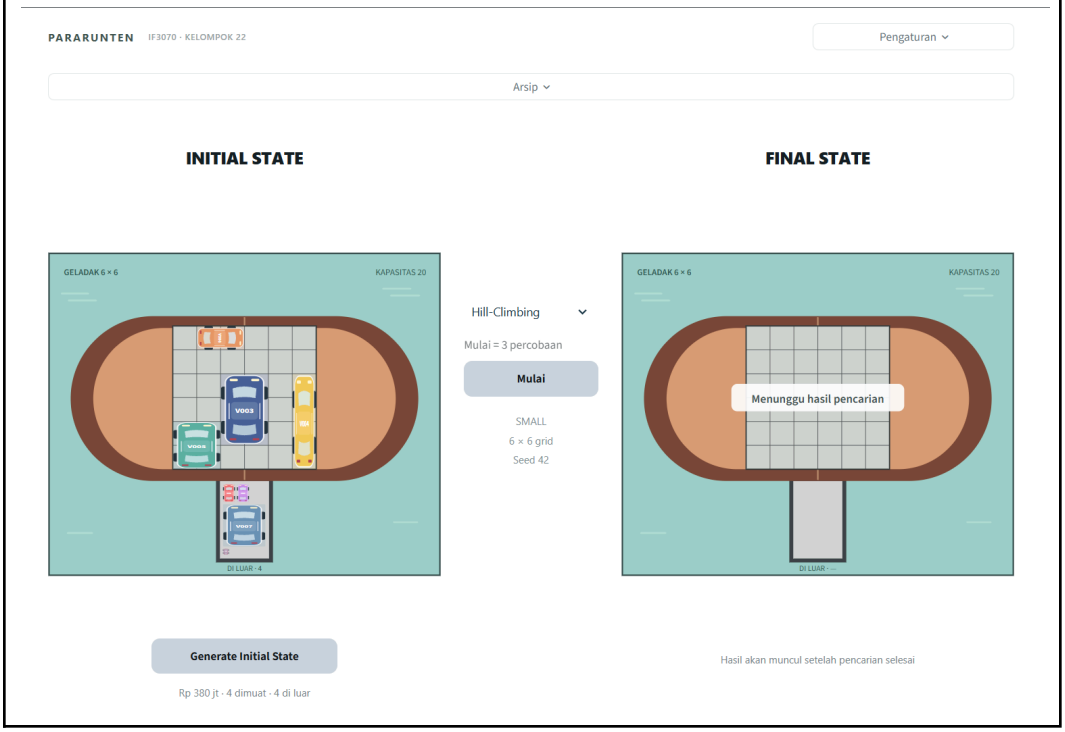

GA A: populasi 20, generasi 200 (1 percobaan)
Maksimum Objective
Rata-rata Populasi
Percobaan 1 (seed 42)
Shipping fee (juta rupiah)
Generasi (0 = populasi awal)
GA A: populasi 20, generasi 50 (1 percobaan)
Maksimum Objective
Rata-rata Populasi
Percobaan 1 (seed 42)
Shipping fee (juta rupiah)
Generasi (0 = populasi awal)
GA B: populasi 10, generasi 100 (1 percobaan)
Maksimum Objective
Rata-rata Populasi
Percobaan 1 (seed 42)
Shipping fee (juta rupiah)
Generasi (0 = populasi awal)

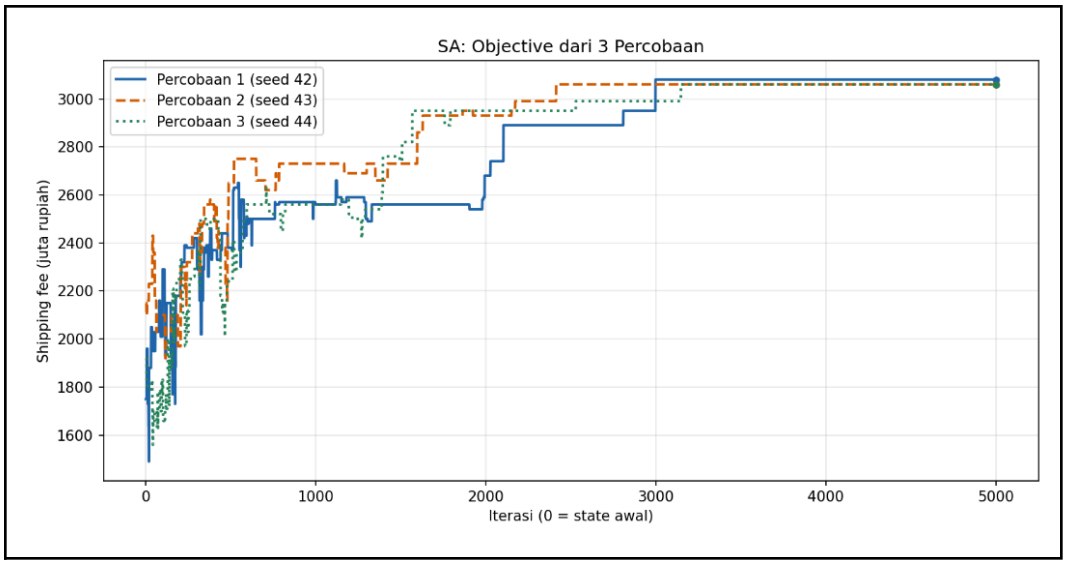




RIngkasan Hasil

```
RINGKASAN SELURUH RUN
HC | Seed 42 | Awal: 1750 | Hasil: 2720 | Iterasi: 107 | Durasi: 1.7267 s
HC | Seed 43 | Awal: 2150 | Hasil: 2750 | Iterasi: 186 | Durasi: 3.2260 s
HC | Seed 44 | Awal: 1920 | Hasil: 2900 | Iterasi: 124 | Durasi: 2.0689 s
SA | Seed 42 | Awal: 1750 | Hasil: 3080 | Iterasi: 5000 | Durasi: 22.5841 s
SA | Seed 43 | Awal: 2150 | Hasil: 3060 | Iterasi: 5000 | Durasi: 28.0321 s
SA | Seed 44 | Awal: 1920 | Hasil: 3060 | Iterasi: 5000 | Durasi: 30.8072 s
GA | Seed 42 | Awal: 2460 | Hasil: 2760 | Iterasi: 50 | Durasi: 3.9756 s
GA | Seed 43 | Awal: 2390 | Hasil: 2690 | Iterasi: 50 | Durasi: 4.0259 s
GA | Seed 44 | Awal: 2360 | Hasil: 2710 | Iterasi: 50 | Durasi: 4.0017 s
GA | Seed 42 | Awal: 2460 | Hasil: 2760 | Iterasi: 100 | Durasi: 7.7995 s
GA | Seed 43 | Awal: 2390 | Hasil: 2690 | Iterasi: 100 | Durasi: 7.7844 s
GA | Seed 44 | Awal: 2360 | Hasil: 2730 | Iterasi: 100 | Durasi: 7.8928 s
GA | Seed 42 | Awal: 2460 | Hasil: 2760 | Iterasi: 200 | Durasi: 15.4784 s
GA | Seed 43 | Awal: 2390 | Hasil: 2710 | Iterasi: 200 | Durasi: 15.6266 s
GA | Seed 44 | Awal: 2360 | Hasil: 2730 | Iterasi: 200 | Durasi: 15.5839 s
GA | Seed 42 | Awal: 2460 | Hasil: 2670 | Iterasi: 100 | Durasi: 3.7059 s
GA | Seed 43 | Awal: 2160 | Hasil: 2670 | Iterasi: 100 | Durasi: 3.5372 s
GA | Seed 44 | Awal: 2310 | Hasil: 2670 | Iterasi: 100 | Durasi: 3.8657 s
GA | Seed 42 | Awal: 2460 | Hasil: 2760 | Iterasi: 100 | Durasi: 7.8116 s
GA | Seed 43 | Awal: 2390 | Hasil: 2690 | Iterasi: 100 | Durasi: 7.7917 s
GA | Seed 44 | Awal: 2360 | Hasil: 2730 | Iterasi: 100 | Durasi: 7.8433 s
GA | Seed 42 | Awal: 2460 | Hasil: 2770 | Iterasi: 100 | Durasi: 15.9328 s
GA | Seed 43 | Awal: 2510 | Hasil: 2920 | Iterasi: 100 | Durasi: 16.3333 s
GA | Seed 44 | Awal: 2360 | Hasil: 2790 | Iterasi: 100 | Durasi: 16.4617 s
```





- **Analisis**

a. Seberapa dekat tiap-tiap algoritma bisa mendekati *global optima* dan mengapa hasilnya demikian?

Algoritma	Objective Terbaik	Persentase terhadap global optimum	Selisih dari global optimum
Hill Climbing	2900	89,23%	350
Simulated Annealing	3080	94,77%	170
Genetic ALgorithm	2920	89,85%	330

Simulated Annealing memperoleh hasil paling dekat dengan global optimum. Kemampuannya menerima penurunan objective sementara memungkinkan pencarian meninggalkan susunan yang sulit diperbaiki secara langsung. Penurunan sementara tersebut dapat membuka ruang untuk memasukkan kombinasi kendaraan dengan total pendapatan lebih tinggi.

Hill Climbing hanya menerima perpindahan yang meningkatkan atau mempertahankan objective. Akibatnya, algoritma lebih sulit menemukan solusi yang membutuhkan pengeluaran kendaraan atau penurunan pendapatan sementara sebelum memperoleh peningkatan.

Genetic Algorithm dapat mengeksplorasi berbagai susunan melalui populasi, crossover, dan mutasi. Namun, pada implementasi ini, individu terbaik tidak otomatis dipertahankan dalam populasi generasi berikutnya. Selain itu, keturunan yang tidak valid diganti dengan individu acak baru, sehingga sebagian susunan yang baik dari induk dapat hilang.

b. Bagaimana perbandingan hasil pencarian tiap-tiap algoritma dengan algoritma *local search* yang lain?

Perbandingan kualitas pencarian dapat dilakukan menggunakan rata-rata objective dari tiga seed.

Algoritma / konfigurasi	Seed 42	Seed 43	Seed 44
HC	2720	2750	2900
SA	3080	3060	3060
GA: populasi 20, generasi 100	2760	2690	2730
GA: populasi 40, generasi 100	2770	2920	2790

Simulated Annealing menghasilkan objective tertinggi pada seluruh seed. Rata-ratanya sekitar 9,92% lebih tinggi daripada Hill Climbing. Perbandingan HC dan SA juga menggunakan state awal yang identik untuk setiap seed, sehingga perbedaan hasil menunjukkan pengaruh proses pencarian setelah inisialisasi.

Genetic Algorithm dengan populasi 40 menghasilkan rata-rata lebih tinggi daripada HC, tetapi selisihnya hanya sekitar 1,31%. Sementara itu, GA dengan populasi 20 dan 100 generasi menghasilkan rata-rata lebih rendah daripada HC.

Meskipun demikian, urutan kualitas tidak selalu sama pada setiap seed. Pada seed 44, HC memperoleh objective 2900 dan mengungguli seluruh konfigurasi GA untuk seed tersebut. Oleh karena itu, hasil satu run belum cukup untuk mewakili kinerja algoritma secara keseluruhan.

c. Bagaimana perbandingan durasi proses pencarian tiap algoritma relatif terhadap algoritma lainnya?

Algoritma	Rata-rata durasi
HC	2,34 detik
SA	27,14 detik
GA: populasi 20, generasi 50	4,00 detik
GA: populasi 20, generasi 100, kelompok A	7,83 detik
GA: populasi 20, generasi 200	15,56 detik
GA: populasi 10, generasi 100	3,70 detik
GA: populasi 40, generasi 100	16,24 detik

Hill Climbing merupakan algoritma tercepat pada eksperimen ini. Simulated Annealing membutuhkan waktu sekitar 11,60 kali lebih lama daripada HC, sedangkan GA dengan populasi 40 dan 100 generasi membutuhkan sekitar 6,94 kali lebih lama daripada HC.

Durasi SA lebih besar karena seluruh run menjalankan 5000 iterasi. Prosesnya juga mencakup pencarian kandidat valid, keputusan

penerimaan, dan pemeriksaan kondisi local optimum setelah perpindahan diterima.

Pada GA, durasi meningkat seiring bertambahnya jumlah individu dan generasi karena semakin banyak keturunan yang harus dibentuk dan dievaluasi.

d. Seberapa konsisten hasil akhir yang didapatkan dari tiap-tiap eksperimen yang dilakukan?

Konsistensi dapat dilihat melalui rentang objective dan standar deviasi sampel dari tiga seed. Standar deviasi yang lebih kecil menunjukkan hasil yang lebih seragam.

Algoritma / konfigurasi	Rentang objective	Standar deviasi sampel
HC	2720–2900	96,44
SA	3060–3080	11,55
GA: populasi 20, generasi 50	2690–2760	36,06
GA: populasi 20, generasi 100	2690–2760	35,12
GA: populasi 20, generasi 200	2710–2760	25,17
GA: populasi 10, generasi 100	2670–2670	0,00
GA: populasi 40, generasi 100	2770–2920	81,45

Simulated Annealing paling konsisten dalam menghasilkan solusi berkualitas tinggi. Selisih hasil tertinggi dan terendahnya hanya 20 poin.

Hill Climbing memiliki variasi lebih besar, dengan rentang 180 poin. Hal ini menunjukkan bahwa seed dan jalur pencarian memberikan pengaruh lebih besar terhadap hasil akhirnya.

GA dengan populasi 10 menghasilkan objective yang sama pada seluruh seed, sehingga standar deviasinya nol. Namun, objective tersebut lebih rendah daripada konfigurasi lainnya. Dengan demikian, hasil yang konsisten belum tentu menunjukkan kualitas solusi yang tinggi. Kesamaan objective juga tidak berarti susunan kendaraannya identik.

GA dengan populasi 40 menghasilkan rata-rata lebih tinggi, tetapi variasi antar-seed juga lebih besar. Peningkatan kualitas pada konfigurasi ini belum diikuti peningkatan konsistensi.

e. **Bagaimana pengaruh banyak iterasi dan jumlah populasi terhadap hasil akhir pencarian pada Genetic Algorithm?**
Pengaruh jumlah generasi diamati pada kelompok A dengan populasi tetap sebesar 20 dan peluang mutasi sebesar 0,1.

Generasi	Seed 42	Seed 43	Seed 44	Rata-rata objective
50	2760	2690	2710	2720,00
100	2760	2690	2730	2726,67
200	2760	2710	2730	2733,33

Penambahan generasi meningkatkan rata-rata objective, tetapi peningkatannya kecil. Dari 50 menjadi 200 generasi, rata-rata objective hanya naik sekitar 0,49%, sedangkan durasi meningkat sekitar 3,89 kali.

Pada seed 42, objective terbaik sudah ditemukan pada generasi ke-23 dan tidak meningkat hingga generasi ke-200. Pada seed 43, peningkatan menjadi 2710 baru terjadi pada generasi ke-148. Sementara itu, seed 44 mencapai nilai 2730 pada generasi ke-87.

Hasil tersebut menunjukkan bahwa tambahan generasi memperbesar kesempatan menemukan solusi lebih baik, tetapi tidak menjamin

peningkatan yang besar. Pada konfigurasi ini, manfaat tambahan generasi semakin kecil dibandingkan pertambahan durasinya.

f. Bagaimana pengaruh jumlah populasi terhadap hasil akhir Genetic Algorithm?

Pengaruh jumlah populasi diamati pada kelompok B dengan jumlah generasi tetap sebesar 100 dan peluang mutasi sebesar 0,1.

Generasi	Seed 42	Seed 43	Seed 44	Rata-rata objective	Generasi
50	2760	2690	2710	2720,00	50
100	2760	2690	2730	2726,67	100
200	2760	2710	2730	2733,33	200

Penambahan populasi memberikan peningkatan kualitas yang lebih besar daripada penambahan generasi dalam rentang parameter yang diuji. Dari populasi 10 menjadi 40, rata-rata objective meningkat sekitar 5,87%, sedangkan durasi meningkat sekitar 4,39 kali.

Populasi yang lebih besar menyediakan lebih banyak individu awal dan kandidat keturunan. Hal ini memperluas peluang menemukan kombinasi kendaraan yang menguntungkan. Namun, peningkatan populasi juga menambah jumlah evaluasi, sehingga manfaat keragaman populasi belum dapat dipisahkan dari manfaat tambahan pekerjaan komputasi.

Populasi 40 memberikan hasil GA terbaik, yaitu 2920 pada seed 43. Akan tetapi, variasi hasilnya lebih besar daripada populasi 10 dan 20. Dengan demikian, populasi lebih besar meningkatkan rata-rata kualitas pada eksperimen ini, tetapi belum menjamin hasil yang lebih konsisten.

Kesimpulan dan Saran

1. Kesimpulan

Berdasarkan hasil eksperimen, algoritma *Hill Climbing, Simulated Annealing,* dan *Genetic Algorithm* memiliki karakteristik yang berbeda dalam menyelesaikan permasalahan penyusunan kendaraan pada kapal. Perbedaan tersebut dapat dilihat dari kualitas solusi yang dihasilkan, konsistensi antara percobaan, serta waktu yang dibutuhkan. Algoritma *Hill Climbing* menghasilkan *objective* terbaik sebesar 2900 atau 89,23% dari global optimum, dengan rata-rata 2790 dari tiga seed dan durasi rata-rata 2,34 detik, menunjukkan bahwa Hill Climbing memiliki keunggulan dari segi kecepatan, tetapi variasi hasilnya masih terlihat cukup besar, dengan rentang *objective* 2720–2900. Pada algoritma Simulated Annealing, dihasilkan *objective* terbaik sebesar 3080 atau 94,77% dari global optimum, dengan rata-rata 3066,67, rentang 3060–3080, dan durasi rata-rata 27,14 detik, menunjukkan bahwa algoritma ini menghasilkan solusi yang relatif konsisten pada pengujian yang dilakukan.

Sementara itu, *Genetic Algorithm* memperoleh objective terbaik sebesar 2920 atau 89,85% dari global optimum. Pada konfigurasi populasi 40 dan 100 generasi, rata-rata objective mencapai 2826,67 dengan rentang 2770–2920 dan durasi rata-rata 16,24 detik. Hasil *Genetic Algorithm* juga menunjukkan bahwa perubahan parameter populasi dan jumlah generasi mempengaruhi kualitas solusi serta waktu komputasi.

Secara umum, hasil eksperimen memperlihatkan adanya pertukaran antara kualitas solusi, konsistensi, dan efisiensi komputasi. Algoritma dengan waktu komputasi lebih singkat belum tentu menghasilkan objective tertinggi dan peningkatan parameter pencarian dapat meningkatkan kualitas solusi dengan konsekuensi bertambahnya waktu komputasi. Pada Genetic Algorithm, peningkatan jumlah

generasi dari 50 menjadi 200 meningkatkan rata-rata objective dari 2720 menjadi 2733,33, sedangkan peningkatan populasi dari 10 menjadi 40 meningkatkan rata-rata objective dari 2670 menjadi 2826,67. Namun, peningkatan tersebut tidak selalu menjamin hasil yang lebih baik pada setiap percobaan. Hasil ini masih perlu ditafsirkan dengan mempertimbangkan keterbatasan eksperimen, yaitu penggunaan satu dataset dan tiga seed untuk setiap konfigurasi, serta perbedaan beban komputasi antar algoritma.

2. **Saran**

Untuk pengembangan selanjutnya, eksperimen dapat dilakukan menggunakan lebih banyak seed dan dataset dengan ukuran serta karakteristik berbeda agar perbandingan kinerja lebih representatif. Pengujian parameter juga dapat diperluas untuk melihat pengaruh strategi pemilihan tetangga pada Hill Climbing, suhu awal dan laju pendinginan pada Simulated Annealing, serta ukuran populasi, jumlah generasi, crossover, dan mutasi pada Genetic Algorithm.

Pembagian Tugas

Nama	NIM	Tugas
Ahmad Aditya Ar Rasyid	18224023	- Pembuatan baseline untuk semua algoritma - Pembuatan algoritma Genetic Algorithm - Menyusun laporan
Daniel Benaya Toar Supaath	18224027	- Pembuatan visualisasi - Pembuatan algoritma Hill-Climbing with Sideways move - Menyusun laporan
Ahmad Rizal Fahmi	18224085	- Pembuatan algoritma Simulated Annealing - Menyusun Laporan

Referensi

https://www.baeldung.com/cs/genetic-algorithms-roulette-selection

https://youtu.be/nhT56blfRpE?si=fBXHuBD7NUNll9Av

Beisiegel, B., Kallrath, J., Kochetov, Y., & Rudnev, A. (2005). Simulated annealing based algorithm for the 2D bin packing problem with impurities. https://doi.org/10.1007/3-540-32539-5_49

Baeldung. (n.d.). Simulated annealing. Retrieved October 1, 2024. Tersedia di https://www.baeldung.com/cs/simulated-annealing

GeeksforGeeks. (n.d.). Introduction to hill climbing in artificial intelligence. Retrieved October 1, 2024. Tersedia di https://www.geeksforgeeks.org/introduction-hill-climbing-artificial-intelligence/

An analysis of solutions to the 2D bin packing problem and additional complexities. (2021, August). https://doi.org/10.36227/techrxiv.15262176.v1

A genetic algorithm for a 2D industrial packing problem. (1999, October). Computers & Industrial Engineering.

TutorialsPoint. (n.d.). Genetic algorithms: Mutation. Retrieved October 2, 2024. Tersedia di https://www.tutorialspoint.com/genetic_algorithms/genetic_algorithms_mutation.htm

Russell, S., & Norvig, P. (2020). Artificial intelligence: A modern approach (4th ed.). Pearson.

Lampiran Form Penggunaan Generative AI

SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, kami:
Nama/NIM :

- Ahmad Aditya Ar Rasyid/18224023
- Daniel Benaya Toar Supaath/18224047
- Ahmad Rizal Fahmi/18224085

Fakultas/Sekolah **:** STEI-K
Kode Mata Kuliah **:** IF3070
Nama Mata Kuliah **:** Dasar Inteligensi Artifisial
Judul Tugas/Proposal **:** Tugas Besar 1 IF3070 Dasar Inteligensi Artifisial

Menyatakan bahwa kami menggunakan AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Tidak	
Pembuatan Teks: menggunakan tools seperti **ChatGPT**, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Ya	AI-assisted workflow
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti **ChatGPT**, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya	AI-assisted workflow

Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Tidak	
Lainnya, Jelaskan:		

Kami juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Kamis, 22 Oktober 2026



Ahmad Aditya Ar Rasyid
18224023

Kamis, 22 Oktober 2026



Daniel Benaya Toar Supaath
18224047

Kamis, 22 Oktober 2026



Ahmad Rizal Fahmi
18224085

Lampiran

Tautan *repository* GitHub
https://github.com/Ch4lalala/IF3070_G22_pararunten/tree/main